FENS 402 ENGINEERING DESIGN PROJECT II

MULTIMODAL MIXTURE OF EXPERTS (MOE) FOR VISUAL QUESTION ANSWERING (VQA)

Project Advisor: Assoc. Prof. Dr. Taner Arsan

Project Team: Mehmet Kuzucu, Şafak Gün

Submission Date: 25.05.2025

Faculty of Engineering and Natural Sciences

Kadir Has University

Spring 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our graduation project advisor, Assoc. Prof. Dr. Taner Arsan, for his invaluable guidance and support throughout our graduate project. We also extend our special thanks to PhD student H. Fuat Alsan for his significant contributions and assistance.

We are also deeply thankful to all our professors, friends, and family members who have supported us throughout our academic journey. Their encouragement and support have been essential to the completion of this project.

ABSTRACT

This project examines the development of a multimodal Visual Question Answering (VQA) system that integrates computer vision and natural language processing to answer questions about images. Utilizing pretrained vision encoders, language models, and fusion mechanisms, the system is designed to understand and reason over both visual and textual inputs. The integration of these components enables the model to handle a wide range of question types, from simple object recognition to complex reasoning tasks that require deeper contextual understanding. Various datasets, including VQA v2, GQA, and LLAVA150K, were used to train and evaluate the model's ability to generalize across different question types and image domains. These datasets were chosen for their diversity and complexity, providing a robust benchmark for assessing the system's performance in real-world scenarios. In addition, the Mixture of Experts (MoE) architecture was analyzed. Due to hardware resource limitations, the full potential of MoE could not be revealed. In addition to model development, a user-friendly web application was implemented, utilizing a React frontend, Spring Boot backend, and Python-based model server, allowing interactive querying with uploaded images. This platform enables users to interact with the system in real time, test its capabilities, and receive visual and textual responses instantly, making the research more accessible and demonstrable. The project demonstrates the challenges and opportunities in building scalable, compact, and efficient multimodal AI systems and demonstrates the baseline for future research in more compact and efficient multimodal understanding systems. We chose this project to explore the multimodal AI field, construct compact and modular architecture, and contribute to research in this field. The results emphasize the feasibility of building such systems within limited resources and provide ideas for optimizing architecture and dataset selection. Future work may focus on refining the MoE framework, expanding dataset diversity, and improving the system's inference speed and accuracy.

TABLE OF CONTENTS

1 INTRODUCTION

In recent years, there have been advances in the combination of computer vision and natural language processing [1]. These advancements led to AI models to interpret and analyze both visual and textual data together. One of the most significant tasks in this domain is Visual Question Answering (VQA). In VQA tasks, the goal is to answer the question based on the content of a given image. This task has gained popularity in recent years due to its broad usage in several areas such as digital assistants, accessibility technologies for visually impaired users, and multimodal search engines [2].

This project focuses on the development, training, and testing of a VQA model. The model has 3 main parts: a pretrained vision encoder, a language model, and a fusion mechanism to integrate visual and textual elements. There are 3 main datasets used in this project: VQA v2 [3], GQA [4], and LLAVA150K [5].

In addition to the model development, a web-based application was built to help users interact with the VQA system in a simple and accessible way. Thanks to this interface, users can upload an image, type a question related to that image, and receive an answer generated by the model. The application was built using a layered architecture, with a React frontend for user interaction, a Spring Boot backend for handling API requests, and a Python-based model server for executing the VQA model.

1.1 Motivation

Multimodal AI models have become increasingly powerful and popular due to the success of large-scale language and vision models in the last 3 years [6]. Its use cases are huge, including assistive technologies for the visually impaired, autonomous robotics, educational tools, and intelligent digital assistants [2]. Moreover, transformer based vision models such as Vision Transformers (ViTs) [7] have shown a strong performance in feature extraction and image classification tasks. Similarly, autoregressive language models such as GPT [8] and LLaMA [9] have shown extraordinary capabilities in natural language understanding and generation.

Traditional VQA models relied on feature concatenation or simple multimodal pooling to combine visual and textual information. While they are functional, these methods struggle to capture deep semantic understanding between modalities. Recent developments have shown that cross attention mechanisms can more effectively and meaningfully combine images and text [10] [11]. However, it should be noted that despite these architectural improvements, training a VQA model is still very challenging because of the fact that these two modalities are fundamentally different in nature [12].

However, state-of-the-art VQA models are often incredibly large and computationally intensive, some of them using billions of parameters and powerful hardware setups such as GPU clusters to train and perform inference. This creates a gap between academic research and practical implementations [13].

This project is motivated to bridge this gap by building an efficient and lightweight VQA model that can work under significant computational constraints. Due to the limitations in both computational power and available resources, this project does not directly compete with the state-of-the-art models. On the other hand, it will focus on studying how far VQA performance can be pushed within a modest, accessible setup.

1.2 Problem Definition

The core problem to be dealt with in this project is: Given an image and a related natural language question, generate an accurate and appropriate answer.

Solving this problem requires a model capable of:

- Understand visual context,
- Interpreting natural language,
- Aligning and fusing multimodal information.

While many recent VQA systems achieve high capabilities using large scale models, these models often rely on billions of parameters and require incredible computational resources [14] [15], including multiple high-end GPUs for both inference and training. Considering these limitations, this project operates under significant computational constraints. Therefore, this project aims to develop such a VQA model that:

- Is computationally efficient,
- Has a low parameter count,
- Requires minimal training data.

This situation introduces additional technical challenges to the project. Some of these challenges are limited capacity and modality alignment. Moreover, since many models benefit from "overparameterization" to avoid underfitting to achieve better results, our model will lack this benefit.

2 RELATED WORK

In this section, we review the foundational work and recent advancements related to our graduation project, focusing on Visual Question Answering (VQA), Mixture of Experts (MoE) architectures, and multimodal learning. We also point out the existing challenges and gaps in the field that motivate our project. Lastly, we discussed our research questions that guide our project.

2.1 Visual Question Answering (VQA)

Visual Question Answering (VQA) is the one of the most important task in the multimodal AI field [1]. It was established with the release of the VQA v1 dataset [1]. Moreover, it is later improved when the VQA v2 [3] dataset is released, which reduced the language bias through balanced question-answer pairs.

Initial models trained for this task used Convolutional Neural Networks (CNNs) such as ResNet [16] for image encoding and Recurrent Neural Networks (RNNs) (e.g., LSTMs) [17] for text encoding. The initial models used relatively simple fusion methods [11]. After the development of attention mechanisms [18], methods such as Bottom-Up and Top-Down Attention model [19] and BAN (Bilinear Attention Network) [20] became more dominant in the model's architecture.

2.2 Mixture of Experts (MoE)

Mixture of Experts (MoE) architecture aims to improve the model's capacity while maintaining a balance on the computational costs. It was first introduced in the paper titled Adaptive Mixtures of Local Experts [21]. Later, this topic was revived in the paper named Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer [22] with the Sparsely-Gated MoE used in language modeling within Google's GShard architecture [23]. In recent years, modern versions such as Switch Transformers have emerged [24]. These days, MoE models demonstrate the ability to scale the models to hundreds of billions of parameters.

2.3 Multimodal Learning

Multimodal learning aims to build models that can join multiple input modalities together, such as vision and language. Some of the key challenges include representation alignment, cross-modal fusion, and generalization across modalities.

Earlier multimodal transformers include:

- ViLBERT [10]
- LXMERT [25]
- VisualBERT [26]
- UNITER [27]

These models introduced various techniques for the alignment and integration of visual and textual modalities. They often used co-attention mechanisms or joint embedding spaces. In particular, cross attention where one modality, such as text, attends to another modality, such as image, has proven to be effective. This technique allows better interaction between modalities and is a key building block in our project's fusion mechanism.

2.4 Gap Analysis

Despite the fast achievements in the VQA and multimodal learning, to this day still, most state-of-the-art models are computationally demanding, often requiring large-scale datasets, powerful hardware, and extensive training time. Since these models are typically developed by large tech companies, they prioritize performance over accessibility. This limits the usage of these models in low-resource environments such as personal projects, academic settings, or smaller institutions.

This project aims to address this gap by focusing on the resource constraints. It aims to explore how far a minimal, yet modular architecture can go in terms of VQA performance without relying on overparameterization or large-scale infrastructure.

2.5 Research Questions

The primary objective of this project is to investigate how resource efficient Visual Question Answering (VQA) models can deliver reasonable performance without using large-scale computational infrastructure or overparameterized models. The following research questions and corresponding hypotheses guide the scope of the project:

- **Research Question 1**

To what extent can a lightweight and compact VQA model perform the task effectively without the use of large-scale infrastructure or extensive computational resources?

- **Hypothesis 1**

We hypothesize that even with limited computational resources and minimal parameter count, it is possible to develop a VQA model that produces reasonable results.

- **Research Question 2**

Can pretrained vision and language components be effectively integrated using a minimal, modular architecture without extensive fine-tuning?

- **Hypothesis 2**

We hypothesize that using existing pre-trained models with minimal architecture and a simple fusion mechanism can lead to a VQA model capable of basic multimodal reasoning.

- **Research Question 3**

How does the model's architecture complexity affect the performance of VQA models?

- **Hypothesis 3**

We hypothesize that in traditional settings, a more developed complex model could better capture the meaning of the modalities.

3 METHODOLOGY

This section describes the overall design and internal structure of the proposed Visual Question Answering system. It explains how visual and textual modalities are processed separately and then fused using cross attention mechanism to generate vision aware outputs. The model architecture consists of image encoder, text encoder, fusion mechanism and LLM. Each component chosen with computational efficiency and modularity in mind.

In addition to the model development, a web-based application was developed to make the model easily accessible. The frontend, built with React, allows users to upload images and ask questions, while the backend, implemented using Spring Boot, handles communication with the Python-based model server, enabling real-time interaction with the VQA system.

The model development is generally divided into two main stages. The first stage is talked about in Section 3.1. The first stage focuses on fine-tuning a pretrained language model on only text based question answering datasets. The second stage, detailed in Section 3.2, talks about vision-language fusion through a cross-attention mechanism. These stages form a complete training system that aims to develop a VLM. The last stage focuses on web-based application development part to create a user friendly environment for users.

3.1 Stage 1: Pretrained LLM Textual Fine-Tuning

To fine-tune our pretrained language model for the first stage of our VLM model, we started with the text-only instruction tuning. This is a necessary step because the language model is pretrained. This means that the model has no capability of answering the questions. It only knows how to do next token generation, which does not provide natural responses. Thus, there is a need for text only fine-tuning to make the model get used to natural responses.

3.1.1 Datasets

In this section, we describe the text-only instruction tuning phase. This section also provides a basis for improving the model's language understanding capabilities before multimodal training. We used three datasets: Databricks Dolly 15k [28], SQuAD v2.0 [29], and TriviaQA [30]. Each

dataset was carefully selected to contribute distinct characteristics, Dolly 15k provides instruction-following behavior, SQuAD v2.0 introduces context-based question answering with unanswerable cases, and TriviaQA offers diverse open-domain factual questions. All datasets were used in the training phase and subsequently evaluated to assess their individual and collective impact on the model's performance.

Databricks Dolly 15k

The first dataset that is used for the first stage training is the Databricks Dolly 15k. It is a publicly available dataset of 15,000 instruction-following records prepared by Databricks. Each record contains three main fields: an instruction, a context, and a response. It covers a variety of categories such as closed question answering, summarization, and classification. This diverse category of options makes this dataset capable of providing natural language examples that are desired in multimodal understanding.

Each field has its own benefits, these are:

- **Instruction**: the question,
- **Context**: optional background information,
- **Response**: the expected answer from the model.

The context field is not used in this project. This is because the aim of the project is not to focus on the text only generation and the format of our prompts is not designed to take any background information. We loaded the dataset using Hugging Face's[1] datasets library as "dataset = load_dataset("json", data_files="databricks-dolly-15k.jsonl")".

Before training, we performed an analysis of token lengths to better understand the data distribution. The dataset ranged from 7 to 8711 tokens, with a mean of about 103 tokens and a median of 68. Most samples were well below 500 tokens, but a small portion (185 samples) exceeded 575 tokens, indicating outliers that could impact training if not handled carefully. The distribution of token lengths can be seen in Figure 3.1.

[1] https://huggingface.co/

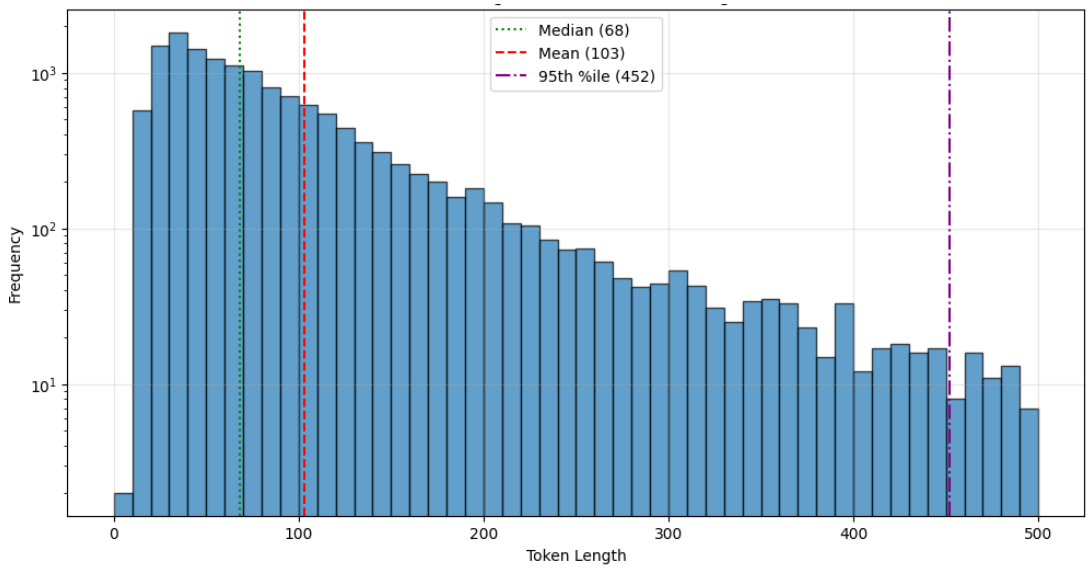

Figure 3.1 Distribution of token lengths of Databricks Dolly 15k

For training, we formatted the dataset into prompt response structure using a special tokenized prompt format. Each prompt was created by combining the instruction and response fields with custom delimiters. Then, these prompts were tokenized using the tokenizer of our selected language model and stored in dictionaries containing input_ids and attention_mask. An example sample from the Databricks Dolly 15k dataset can be seen in Table 3.1.

Table 3.1 Example sample from Databricks Dolly 15k dataset

Instruction	When did Virgin Australia start operating?
Context	Virgin Australia, the trading name of Virgin A...
Response	Virgin Australia commenced services on 31 August 2000 as Virgin Blue, with two aircraft on a single route.
Category	closed_qa
Prompt	<start_sequence><question>When did Virgin Australia start operating?<answer>Virgin Australia commenced services on 31 August 2000 as Virgin Blue, with two aircraft on a single route.<end_sequence>
Tokenized_prompt	[input_ids, attention_mask]

To simplify the training process, we chose to preprocess and store the data in advance rather than processing samples on-the-fly. This decision significantly reduced loading time and allowed us to efficiently train on the dataset using PyTorch[2] DataLoaders. This reduced bottlenecks during model training, too.

SQuAD v2.0

The Stanford Question Answering Dataset (SQuAD) was one of the main datasets used in this stage of our text only model training. It contains around 130,000 question–answer pairs derived from a set of Wikipedia articles. Each entry contains a context paragraph, a question, and one or more annotated answers. The answers are, most of the time, extracted directly from the context. The dataset was originally designed for extractive QA tasks, which means models learn to locate the answers within the context given. However, our aim was not to focus on training a model to extract answers from the context, but rather, to teach a model to generate answers to questions in a human-like style. This required careful transformation of the dataset.

One major challenge was the fact that some questions had no corresponding answer within the context. These samples are useful in the original benchmark, which includes some unanswerable questions to evaluate the model. However, these samples are incompatible with our training aim. Including the no-answer samples would confuse the model during training. To solve this problem, we filtered all samples with empty or null answer fields step by step. This step reduced the dataset size but resulted in a cleaner dataset. Specifically, the number of records decreased from 130,319 to 86,821. After filtering, we structured each sample into our prompt format. Then, the prompts were tokenized. Tokenization was done using the model's tokenizer, and the resulting input_ids and attention_mask were stored alongside the textual data. An example sample from the SQuAD v2.0 dataset can be seen in Table 3.2.

[2] https://pytorch.org/

Table 3.2 Example sample from SQuAD v2.0 dataset

Id	56be85543aeaaa14008c9063
Title	Beyoncé
Context	Beyoncé Giselle Knowles-Carter (/biːˈjɒnseɪ/ bee-YON-say) (born September 4, 1981) is …
Question	When did Beyonce start becoming popular?
Answers	{'text': array(['in the late 1990s'], dtype=object), 'answer_start': array([269])}
Prompt	<start_sequence><question>When did Beyonce start becoming popular?<answer>in the late 1990s<end_sequence>
Tokenized_prompt	[input_ids, attention_mask]

To better understand the dataset, a statistical analysis is done. The results were as follows:

- **Max token length**: 421
- **Min token length**: 9
- **Mean**: ~22 tokens
- **Median**: 21 tokens
- **Standard deviation**: ~6.2
- **95th percentile**: 38 tokens
- **Mean + 2 std dev**: ~34.5
- **Mean + 3 std dev**: ~40.7
- **Samples exceeding 512 tokens**: 0

This distribution shows that most of the inputs were well below the typical transformer model input limit (512 tokens), as can be seen in Figure 3.2.

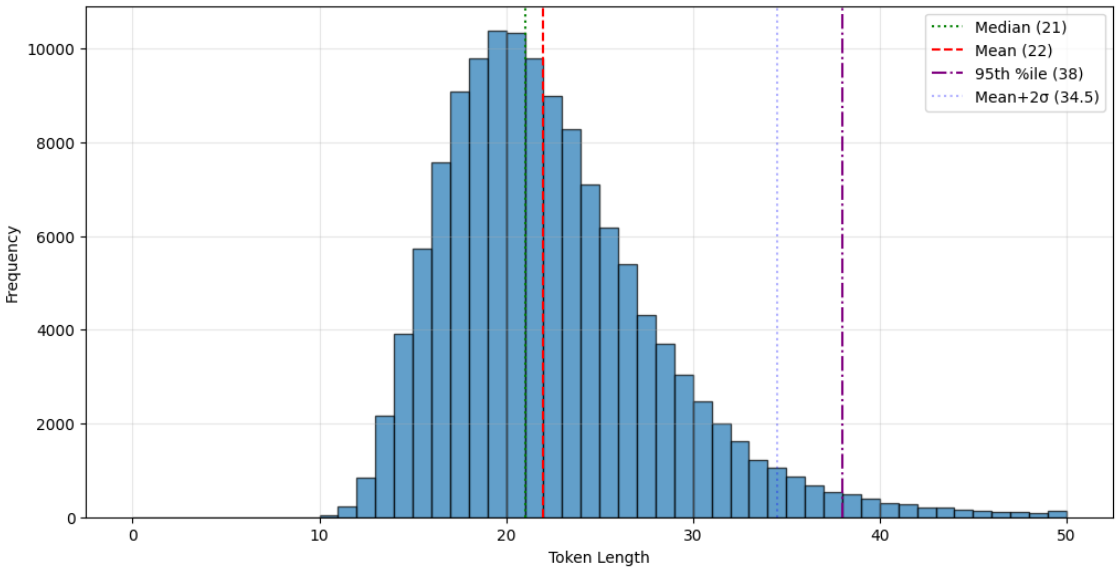

Figure 3.2 Distribution of token lengths of SQuAD v2.0

TriviaQA Dataset

The final dataset used in this stage was the TriviaQA dataset. It is a large-scale question–answering dataset that focuses on trivia-based questions. Unlike other datasets that focus more on the language style answers, TriviaQA is focused on handling general knowledge and world facts, such as "Who was the 5th president of the United States?" or "In what year was the Eiffel Tower completed?". We loaded the TriviaQA training split from a preprocessed ".parquet" file using the code snippet "dataset_df = pd.read_parquet("train.parquet")".

The loaded dataset contained 138,384 training samples. Each row has the following fields: question, question_id, question_source, entity_page, search_results, and answer. For consistency with our structure, samples are structured according to our prompt structure. We also added a prompt column. We also added a prompt column with the formatted prompt string and a tokenized_prompt column, which included the tokenized version of the prompt using our model tokenizer. This field contained input_ids and attention_mask fields. An example sample from the TriviaQA dataset can be seen in Table 3.3.

Table 3.3 Example sample from TriviaQA dataset

Question	Which widely known song was composed by one-time docker Jim Connell in 1889 on a No 28 London bus?
Question_id	sfq_2219
Question_source	www.sfquiz.org.uk
Entity_pages	{'doc_source': array([], dtype=object), 'filename': array([], dtype=object), 'title': array([], dtype=object), 'wiki_context': array([], dtype=object)}
Search_results	{'description': array([], dtype=object), 'filename': array([], dtype=object), 'rank': array([], dtype=int32), 'title': array([], dtype=object), 'url': array([], dtype=object), 'search_context': array([], dtype=object)}
Answer	{'aliases': array(['The Red Flag Once A Year', 'The Red Flag (song)', 'The red flag', "The People's Flag is Palest Pink", 'Battle Hymn of the New Socialist Party', 'The Red Flag'])}
Prompt	<start_sequence><question>Which widely known song was composed by one-time docker Jim Connell in 1889 on a No 28 London bus?<answer>The Red Flag<end_sequence>
Tokenized_prompt	[input_ids, attention_mask]

To assess the dataset, the length of the tokenized prompts is analyzed as can be seen in Figure 3.3. The results were as follows:

- **Maximum token length**: 258
- **Minimum token length**: 11
- **Mean token length**: ~27.05
- **Median token length**: 25
- **Standard deviation**: ~10.56

- **95th percentile**: 56
- **Mean + 2 std**: ~48.17
- **Mean + 3 std**: ~58.73
- **Samples >512 tokens**: 0

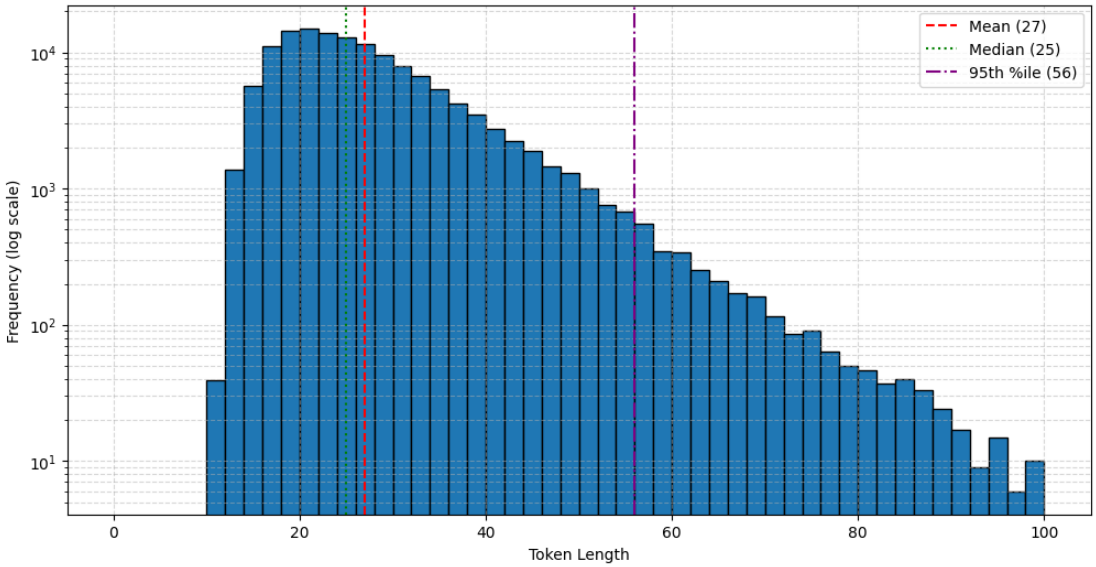

Figure 3.3 Distribution of token lengths of TriviaQA

3.1.2 Language Model Selection and Motivation

The selection of a suitable large language model (LLM) is an important decision for any natural language processing task. It becomes even more important in our multi stage training system. In Stage 1, we focused on selecting a compact, instruction following model that could generalize well to question-answering tasks.

We shaped our selection on several factors:

- **Model Size and Resource Constraints**: Since our training environment had limited resources, we prioritized models under 500 million parameters. Models with small parameters lead to faster experimentation, greater batch sizes, and the possibility of training longer.
- **Open-Source:** Being an open-source model was very important in terms of transparency and reproducibility.
- **Architecture Type:** We were also focused on trying newer architectural trends. For example, Mixture-of-Experts (MoE) architecture offers dynamic routing capabilities.

Although these models are more complex in architecture, we used them for experimental purposes.

We evaluated around ten small to mid-sized transformer-based models. Based on our selection criteria, we selected 3 models for initial training.

HuggingFaceTB/SmolLM2-135M[3]

SmolLM2-135M is a compact LLM optimized for small scale environments. It is specifically designed for experimentation in low GPU environments. It had some key advantages for our project. First of all, it has an extremely small footprint both in terms of parameter size and memory requirements. It is faster to train and has a reasonable capacity. While it is not very good at some of the benchmarks, its small size makes it the best choice for our project.

HuggingFaceTB/SmolLM2-360M[4]

SmolLM2-360M model has a higher-capacity version of SmolLM2 model series. It contains about 2.5x more parameters than the 135M model. Although we performed initial benchmarking, in the end, we chose not to train this model due to increased resource demands. Despite its improved representational power, we observed longer training times and increased usage of memory.

Qwen/Qwen2.5-0.5B [5]

Qwen 2.5 (0.5B version) is an Mixture-of-Experts model developed by Alibaba. It uses sparse routing, which means activating only a subset of its parameters for each input. We wanted to use this model for its MoE architecture and potential modular design. However, we faced several challenges. The training time and memory required increased drastically. Moreover, initial training did not show any model improvements. As a result, we trained Qwen2.5-0.5B only for isolated experimentation and did not proceed with full training.

After careful experimentation and benchmarking, SmolLM2-135M was chosen as the main language model. It offered the best balance between size, efficiency, and performance. Crucially,

[3] https://huggingface.co/HuggingFaceTB/SmolLM2-135M
[4] https://huggingface.co/HuggingFaceTB/SmolLM2-360M
[5] https://huggingface.co/Qwen/Qwen2.5-0.5B

it provided stable outputs during QA-style tuning, creating a strong base for future image-language fusion in Stage 2.

3.1.3 Implementation

The implementation phase involved creating a modular system for the selected language model. Our goal was to keep the training loop minimal but scalable. This allowed us to easily experiment with the different datasets and model precision, as well as optimizer settings.

Precision Testing: Full, Half, and bfloat16

Before deciding on the training settings, we experimented with three different floating-point formats to select the most suitable balance between memory efficiency and numerical stability:

- **Full precision (float32):** Provided a good result but required significantly more GPU memory. This limits the batch size and sequence length.
- **Half precision (float16)**: Provided memory saving and improved the training speed. This precision balanced the quality and the limitations.
- **bfloat16**: Although this precision was promising, bfloat16 was not natively supported on all the hardware setups we used.

Thus, we decided to use float16 (half precision) as our standard training format using torch.cuda.amp mixed-precision utilities.

Model Loading

We loaded our models directly from Hugging Face using the transformers library. Then the model was moved to the target device (cuda) as can be seen in Figure 3.4.

```
model_name = "HuggingFaceTB/SmolLM2-135M"
model = AutoModelForCausalLM.from_pretrained(model_name).to(device)
```

Figure 3.4 Model loading code snippet

Dataset Loading and Preprocessing

For each dataset (e.g., SQuAD, TriviaQA, Dolly), we created a Dataset class to handle the data loading and the tokenization logic. At this point, we used the model's original tokenizer. In Figure 3.5, there is an example for the SQuAD dataset.

```
class SquadDataset(Dataset):
    def __init__(self, pickle_path):
        self.data = pd.read_pickle(pickle_path)
        self.tokenized_prompt = self.data['tokenized_prompt']

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        tokenized_item = self.data.iloc[idx]['tokenized_prompt']

        return {
            'input_ids': tokenized_item['input_ids'].squeeze(),
            'attention_mask': tokenized_item['attention_mask'].squeeze()
        }
```

Figure 3.5 An example SQuAD dataset class code snippet

Each row of the Squad pickle file was preprocessed to include tokenized input fields. Moreover, we used DataLoader from PyTorch with shuffling and batching enabled. The code snippet can be seen in Figure 3.6. This modular system allowed us to easily use the other datasets in this system.

```
dataset = SquadDataset("squad_dataset_preprocessed.pkl")
dataloader = DataLoader(dataset, batch_size=hyperparameters['batch_size'], shuffle=True,
drop_last=True)
```

Figure 3.6 An example dataloader code snippet

Training Loop

Our training loop was implemented from scratch and used PyTorch's basic components. It included gradient computation, backpropagation, and optimizer steps per batch. Below is the simplified version of the training loop can be seen in Figure 3.7.

```python
def train_step(model, batch, loss_fn, optimizer, device):
    model.train()

    input_ids = batch["input_ids"].to(device)
    attention_mask = batch["attention_mask"].to(device)

    outputs = model(input_ids, attention_mask=attention_mask, labels=input_ids)
    logits = outputs.logits
    loss = outputs.loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

Figure 3.7 Training step function code snippet

While the model calculates the cross-entropy loss via labels=input_ids, we also implemented manual label shifting (commented out) to fine-tune the loss calculation for certain experiments. This training step function was used to create epoch based training loop, which loops through epoch times and processes each batch to train the model. We used CrossEntropyLoss and AdamW as our default loss function and optimizer as can be seen in Figure 3.8. Moreover, the whole training process was monitored using tqdm 6 library. This library is minimal but useful to report each batch and epoch's loss.

```python
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
```

Figure 3.8 Loss function and optimizer code snippe

[6] https://tqdm.github.io/

3.1.4 Evaluation of Text-Only Performance

Evaluating the performance of large language models (LLMs), especially in generative tasks, is a significant challenge. Unlike classification tasks with clear ground truths, open ended generation tasks don't have the ground truth answers. This makes the quantitative evaluation both difficult and misleading.

Limitations of Standard Metrics

Traditional automatic metrics such as BLEU [31], ROUGE [32], or METEOR [33] are used in machine translation and summarization. However, they are not very suitable for evaluating open ended generations. These metrics focus on n-gram overlaps between the predicted and reference texts. Since the nature of the generative tasks, these metrics are failing. For example, two logically similar answers can get low scores if they differ in phrasing or structure. This subject will be discussed in section 3.2.4 again.

Because of these limitations, we didn't use any automated metrics in this initial stage. Instead, we made a manual assessment of the model outputs.

Manual Evaluation Strategy

Since the purpose of this stage was not the final deployment of the model but only to identify the strong base LLM for the stage 2 multimodal training, we limited our evaluation to manual assessment of the generated outputs.

The manual evaluation involved:

- Analyzing model generated answers for correctness and fluency with the question.
- Comparing outputs across other models (SmolLM2-135M, SmolLM2-360M, Qwen2.5-0.5B).

This manual selection was sufficient for finding the major weaknesses and selecting the most promising model.

This stage was not used as a final benchmark but just a filtering phase to select the best LLM from other candidates for the second multimodal training stage. Thus, deep benchmarking was deliberately not done.

3.2 Stage 2: Vision-Language Fusion Training

In this stage, we focus on merging the visual and textual modalities. The goal is to enable the model to generate vision aware responses by understanding the relationship between image content. For this purpose, we used vision-language datasets such as VQA v2, GQA, and LLAVA150K. The training process involved fusing image features extracted by the vision encoder with textual representations from the language model using a cross-attention-based fusion mechanism.

3.2.1 Datasets

To train and evaluate our Visual Question Answering (VQA) model, we used there datasets: VQA v2, GQA, and LLaVA-Instruct-150K. These different datasets serve a variety of purposes. VQA v2 and GQA provide real-world question answering pairs with image and textual data, useful for core model training and evaluation. LLaVA-Instruct-150K, on the other hand, offers multimodal instruction-following samples that can be a better choice for getting natural answers. Thus, these datasets will allow us to test our model's architecture with different purpose datasets.

VQA v2

The VQA v2 dataset is one of the most widely used datasets used both in training and benchmarking for testing visual capabilities. It is developed on top of its predecessor, VQA v1. It introduced new balanced question pairs.

The dataset contains over 265,000 images that are sourced from MS COCO dataset [34]. Each image is annotated with at least three questions. The dataset is divided into 3 sets: training, validation, and test sets. The questions span yes/no answers, counting, and open-ended "other" types, which test a broad range of reasoning capabilities, including object recognition, spatial understanding, scene context analysis, and general scene understanding.

In order to use this dataset, a preprocessing step was necessary. Due to the large size of the dataset, it was not practical to include raw images in easy-to-use formats such as parquet or pickle. Instead, the images were stored in folders in JPG format. Since the images are the same images that are in the COCO dataset, there was no need to download a separate dataset. Thus, the first step was to download the COCO images. The first step involved downloading the COCO training

images, which amount to approximately 18GB and include 118,287 images. After downloading the images, the next step was to download the annotations that contained the questions and answer pairs. These annotations are used for various computer vision tasks, not just visual question answering (VQA), so only the relevant columns were extracted for this project. It should also be noted that in the dataset, each question has multiple answers with multiple annotators. To determine the final answer for each question, we selected the most frequently given response as the ground truth.

To improve training efficiency, we made a decision not to process the dataset on-the-fly during the training. Instead, we preprocessed and saved the structured data as a serialized Pickle[7] file for faster access. The reason for this decision was very important because even just matching questions and answers to their corresponding image paths took around an hour, not including any training. In this process, each question from the annotations was paired with the most frequently chosen answer and linked to the relevant image file. The final dataset was saved in a pickle format with columns named question, answer, and image_path, as can be seen in Table 3.4.

Table 3.4 Example sample from Squad dataset

Question	What is this photo taken looking through?
Answer	net
Image_path	C:\Users\mehme\Documents\coco\images\train2017\train2017\ 000000000009.jpg

After completing the preprocessing, a total of 443,757 samples were prepared and ready for training. An example of a sample can be seen in Figure 3.9.

[7] https://docs.python.org/3/library/pickle.html

Figure 3.9 Example sample from VQA v2

GQA

GQA (Graph Question Answering), introduced by Hudson and Manning in the paper named "*GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering*" [4] focuses on full sentence answers rather than short one-word responses. Unlike VQA v2, which includes a mixture of simple and complex questions, GQA is designed to capture scene graph annotations from the Visual Genome dataset [35].

For this project, instead of using the original raw dataset, which includes image files and annotations in a separate file, we preferred a more convenient version hosted on HuggingFace named vikhyatk/gqa[8] dataset. This version has both the question-answer pairs and corresponding images already matched. The format in which the dataset is stored is Parquet format. The dataset was divided into 21 Parquet files, each approximately 500MB in size, which adds up to approximately 10 GB in total. Each Parquet file contains two columns: one for the image, which is stored in raw binary format, and another containing a list of question-answer pairs. Question-answer pairs in the dataset include not just short answers but also more descriptive full answers. This integrated format of the dataset helped with the preprocessing as the image and the question-answer pairs were already matched. This feature eliminated the need for a manual matching between images and the annotations, which was the necessary step in the VQA v2 preprocessing.

[8] https://huggingface.co/datasets/vikhyatk/gqa

Working on this dataset required a slightly different approach compared to VQA v2. For example, reading Parquet files was more complex than handling the standard formats like CSV or JSON due to the fact that Parquet files are more compressed, and the images are stored in a binary format. Thus, reading the dataset required higher computational resources during the loading phase. To handle the data, we used the "Pandas[9]" framework for reading the Parquet files. Moreover, we loop over the dataset to iteratively read and merge all 21 files into a single DataFrame. Since the image data was stored as raw binary, in order to decode them, another library called "io[10]" is used. After processing, each record in the dataset contained four fields: image, question, answer, and fullAnswer, as can be seen in Figure 3.10. This structure allowed us to directly access both images and the question-answer pairs in the training steps. At the end of preprocessing, a total of 1,075,062 samples were prepared. An example sample can be seen in Table 3.5. However, it should be noted that due to the large number of samples in the dataset, training on the full dataset becomes a significant challenge in terms of memory and computational requirements. These limitations and the solutions will be discussed in detail in the experimental section.

Table 3.5 Example sample from GQA dataset

Image	<PIL IMAGE>
Question	Is the sky dark?
Answer	yes
FullAnswer	Yes, the sky is dark.

[9] https://pandas.pydata.org/
[10] https://docs.python.org/3/library/io.html

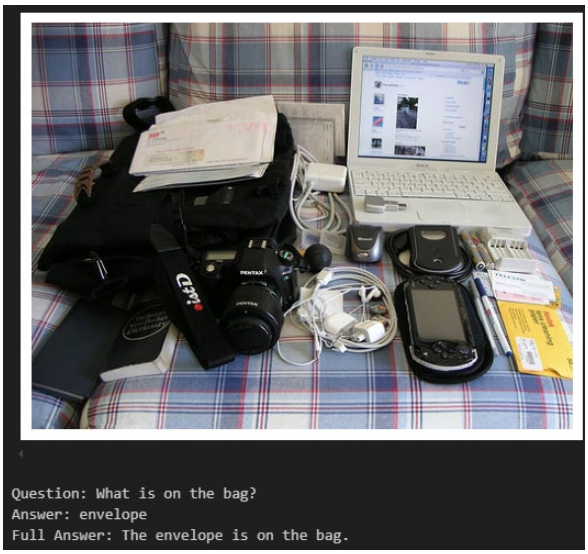

Figure 3.10 Example sample from GQA

LLaVA-Instruct-150K

The LLaVA Visual Instruct 150K dataset approaches the development of multimodal models in a new trend. Unlike earlier academic datasets such as VQA v2 and GQA, which are specifically structured for research objectives, this dataset focuses on open-ended, and instruction-based question-answer pairs that aim to be more natural and conversational in their usage of the language. LLaVA simulates example scenarios where the model is expected to give coherent and natural responses to the questions provided with visual inputs. These responses often go beyond simple one phrase answers, and instead aim to mimic the real human language. This makes the model to become more like a large language model.

The LLaVA-Instruct-150K dataset required a different processing strategy compared to earlier academic datasets due to its focus on open-ended, natural language responses. The dataset is structured in a way that each record has an image identifier, which is a COCO image id. Another column of the dataset was the conversations column. The Conversations column consists of conversational turns between a user ("human") and the assistant model ("gpt"). Each conversation typically contains one instruction-like input and a corresponding detailed response. Since the images are the references of the COCO dataset, which at this point we already have locally, we did not re-download them, only annotations were downloaded at this point. The id references then used to construct image paths by matching the ids with the corresponding COCO image. This early mapping allowed us to access only the necessary images without using the whole COCO dataset.

As a next step, question-answer pairs are extracted and used in our own prompt structure. This prompt structure will be discussed in the Methodology section. The resulting dataset has 4 fields: id, image, conversions, and prompt, as can be seen in Figure 3.11. After the processing of the whole dataset, a total of 157,712 samples were ready for training. An example sample can be seen in Table 3.6.

Table 3.6 Example sample from LLaVA-Instruct-150K dataset

Id	33471
Image	C:/Users/mehme/Documents/coco/images/train2017/train2017/000000033471.jpg
Conversations	[{'from': 'human', 'value': '<image>\nWhat are the colors of the bus in the image?'}, {'from': 'gpt', 'value': 'The bus in the image is white and red.'}, {'from': 'human', 'value': 'What feature can be seen on the back of the bus?'}, {'from': 'gpt', 'value': 'The back of the bus features an advertisement.'}, {'from': 'human', 'value': 'Is the bus driving down the street or pulled off to the side?'}, {'from': 'gpt', 'value': 'The bus is driving down the street, which is crowded with people and other vehicles.'}]
Prompt	<start_sequence><question>What are the colors of the bus in the image?<answer>The bus in the image is white and red.<end_sequence>

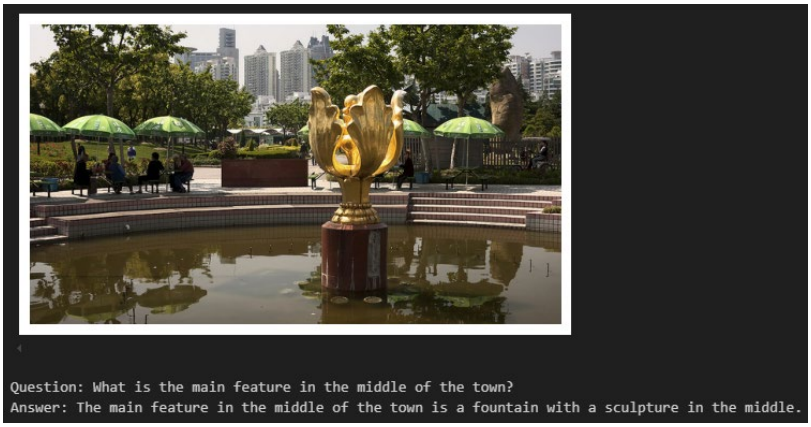

Figure 3.11 Example sample from LLaVA-Instruct-150K

3.2.2 Model Architecture

This section describes the architecture of our multimodal Visual Question Answering (VQA) model. The model is designed to effectively merge visual and textual inputs to generate meaningful and relevant answers. The model has 4 main components: a vision encoder, a language model, a textual encoder, and a lightweight fusion mechanism as can be seen in Figure 3.12. The overall design uses a modular design, allowing each component to be evaluated and, if needed, trained by itself. Moreover, this modular design can allow the different components to be independently replaced. Since we will use pretrained backbones, this feature is crucial for us.

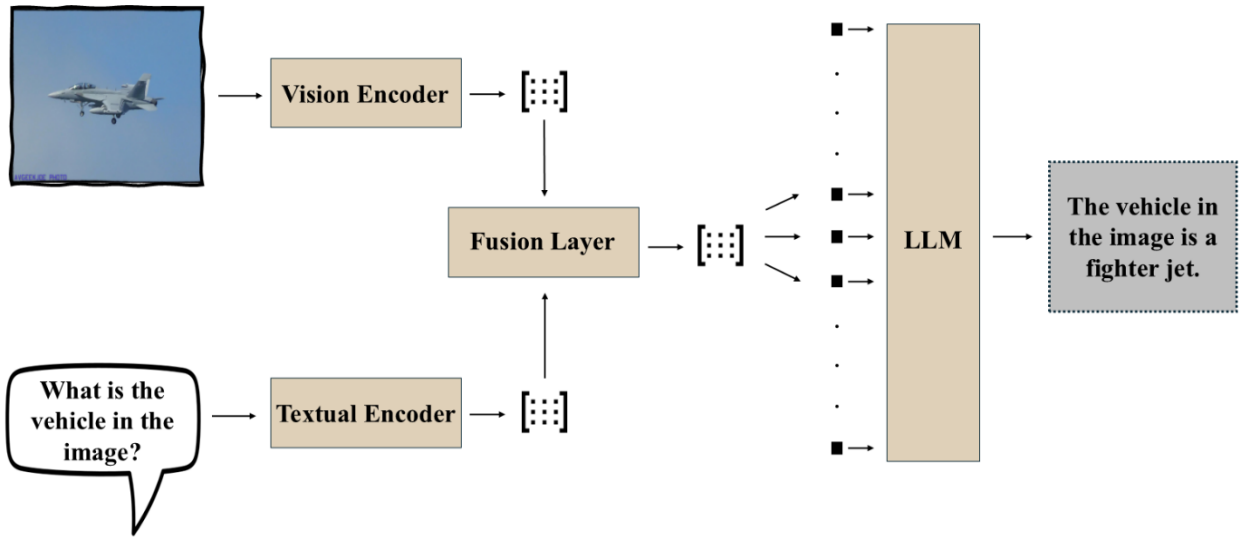

Figure 3.12 Overall design of the Vision-Language Model (VLM)

Each component in the model has its own purpose. The vision encoder is responsible for extracting visual features from input images, while the textual encoder is responsible for processing the input question. The fusion module's main purpose is to enable cross-modal interaction between the two modalities. Lastly, the language model (LLM) is used as a decoder block to produce the final textual answer.

Vision-Encoder (Visual Feature Extraction)

In the design of multimodal architectures for Visual Question Answering (VQA), one of the most important components is the vision encoder. The vision encoder's role in the model is to extract meaningful representations from raw input images. This representation is crucial for models to understand the visual inputs. There are two strategies for using the vision encoder in the VQA

model. The first strategy is to train a custom vision encoder from scratch, typically using convolutional or transformer-based architectures [36]. The second strategy, on the other hand, relies on utilizing an existing pre-trained model. This pretrained model can be used as a frozen or partially trainable backbone, depending on the tasks and design choices of the model. Considering the limitations and feasibility of the project, we decided to choose the second strategy and use a pretrained model as a visual encoder backbone. This decision matches many other projects in this field. Most of the models in this field commonly employ a visual encoder as their backbone.

Our decision not to train the vision encoder from scratch was driven mainly by consideration of feasibility and resource efficiency. Training the vision model from scratch, specifically the ones with the modern deep learning architectures such as Vision Transformers (ViTs) or deep residual networks (ResNets), requires using huge, annotated datasets and significant computational resources. Moreover, since this academic project is not funded by any institution. Thus, after careful consideration of the constraints that involve limited training time, data access, and computational power, training a vision encoder was considered impractical.

Instead, we chose to use a pretrained Vision Transformer (ViT) model, specifically the vit_b_16[11] variant with weights obtained from the IMAGENET1K_SWAG_LINEAR_V1 checkpoint. This model was trained on various datasets, mainly from the SWAG dataset, which is the ImageNet variant. There are other variants of vit_b_16, however, IMAGENET1K_SWAG_LINEAR_V1 is chosen for our model because of its success on benchmarking tasks. The ViT model architecture proposes several advantages over the traditional convolutional network architecture. Traditional convolutional networks like ResNet extract the local features through stacked convolution layers. On the other hand, VITs use self-attention, which helps the model to capture long-range understanding over the text. This architectural difference allows ViTs to have a complex reasoning where understanding the relationship between objects is critical in different regions of the image.

Although there are alternatives to ViTs, such as ResNet-50[12] or ResNet-152[13], which also have high performance, we choose to use the ViT architecture because of its compact size and

[11] https://docs.pytorch.org/vision/main/models/generated/torchvision.models.vit_b_16.html
[12] https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html
[13] https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet152.html

transformer-based design. Furthermore, using a relatively small ViT model, vit_b_16 as a backbone helped to keep computational resource needs in balance.

The ViT encoder block also includes a predefined transformation pipeline, which preprocesses the raw input images into the desired format. This process includes resizing, normalization, and tensor conversion steps.

Finally, it should be noted that there are two ways to get the resulting feature embeddings, one of them is single level embeddings and the other is a full sequence of patch embeddings. Although both of these types are used at the beginning of the project, the single level embeddings are chosen to be used in further training because of their success in early training steps.

In summary, our vision encoder block is designed to provide a compact but powerful representation of the visual inputs, using a transformer-based architecture. As illustrated in Figure 3.13, which displays the general design of the vision encoder block, by using a pretrained ViT model as a frozen backbone, we achieve a balance between computational efficiency and powerful architecture for multimodal learning.

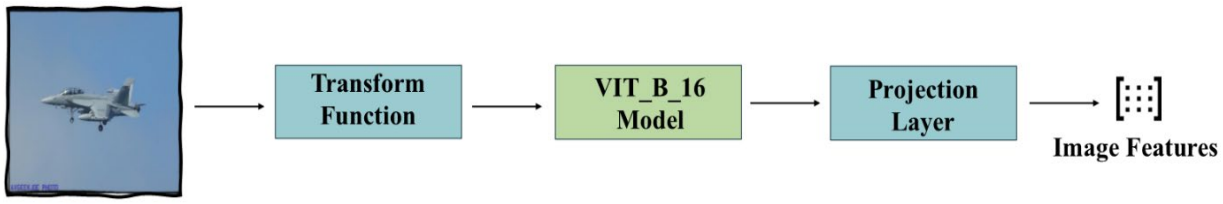

Figure 3.13 General design of the vision encoder

Textual-Encoder

The textual encoding is the second component of our multimodal architecture. This component is responsible for processing the input textual question into a sequence of vector representations. These representations will later be fused with visual features extracted from the image encoder. In contrast to the vision encoder, which is set to be frozen during training of the model, the textual encoder is a fully trainable component. Its parameters are updated along with the rest of the model during the training phase.

Rather than training the textual encoder from scratch, we use the token embedding layer of a pretrained transformer-based language model. The reason for taking this decision is driven by both practical and theoretical considerations. Training a textual encoder from scratch would require an

extremely large corpus of text data. Moreover, there would be a need to have a considerable computational resources. Thus, using the LLMs embeddings layer, we can easily get the representation capability we need.

There are several alternatives we considered for the textual encoder. Traditional sequence models such as Long Short-Term Memory (LSTM) networks have been used in the past in VQA models. However, these architectures fail to capture long range understanding of the given texts. Thus, they are not utilized for this project.

In conclusion, the textual encoder in our model is a purpose-specific module that transforms the textual question into token embeddings using the LLM's embedding layer. It is trained along with the rest of the VQA model to adapt dynamically to VQA tasks.

Fusion Architecture Design

The fusion block is one of the most important components of any vision language model. Because it is responsible for the interaction between visual and textual modalities. Its goal is to combine the extracted image features with the textual input representations in such a way that both modalities do not lose the information. In our model architecture, we utilized a Cross-Attention mechanism to successfully merge the visual embeddings produced by the vision encoder and the token-level embeddings generated by the textual encoder.

As its core, cross-attention allows the question tokens (queries) to attend to image features (keys and values). This attention mechanism blends the two representations such that both linguistic and visual information keep its meaning. The approach is used by other transformer-based models such as ViLBERT [10] and LXMERT [25].

We implemented this mechanism using PyTorch's MultiheadAttention module. Each token in the question sequence independently attends to the set of image embeddings. This allows the model to determine which parts of the image are most relevant to each token in the question. This dynamic process makes the merging of the two modalities versatile, as there is no hard coded interaction flow; on the contrary, the whole process is learned during training. The output of this attention operation is then added to the original textual token embeddings using a residual connection and a normalization layer step. These actions are taken to keep the model stable during the training. The overall design of our fusion mechanism can be seen in Figure 3.14.

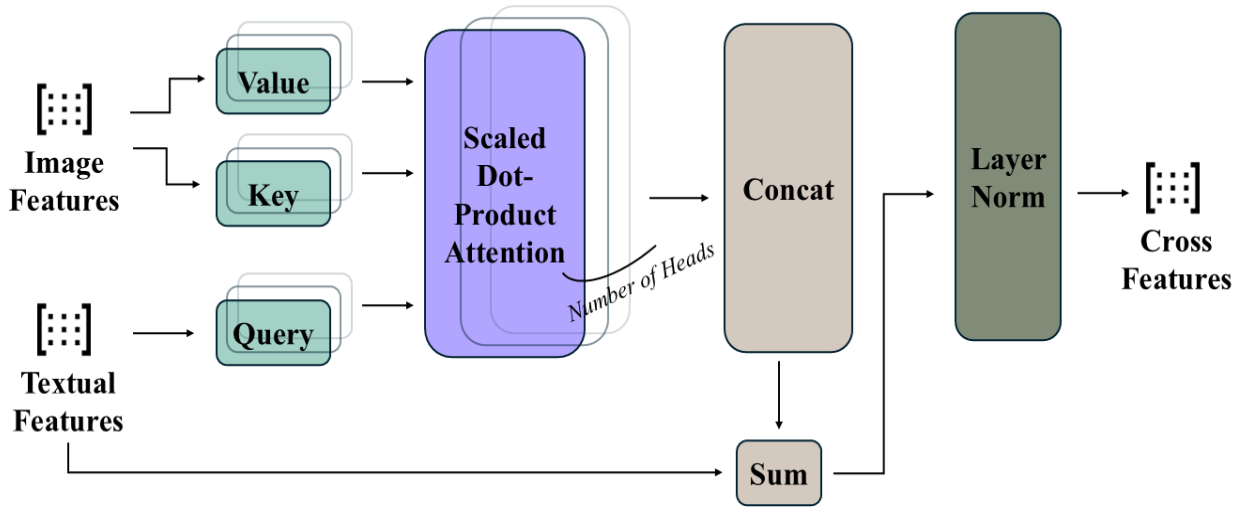

Figure 3.14 General design of the fusion mechanism

This cross-attention method helps match the word to specific parts of the visual input. For example, in theory, the word ‘object’ will focus on the parts of the image where the objects are located. Unlike the other methods, such as early concatenation-based methods, which just combine the image and sentence into a single vector, cross-attention keeps the details separate. This separation makes it possible to reason about individual entities, relationships, and attributes.

We chose a lightweight architecture in terms of depth and parameter count, only using a single layer of multi-head attention rather than stacking two or more attention blocks. This decision was made the balance the computational efficiency to limit the training resources. However, it should be noted that despite its simplicity, this single layer design demonstrated a successful result.

However, to explore and test the performance, we also implemented a significantly more complex bidirectional fusion model. This alternative version used multiple layers of bidirectional cross-attention, conditional feedforward networks, and layer-wise fusion between a Vision encoder and a transformer-based language model. In this architecture, the visual and textual representations go through iterative refinement through stacked bidirectional attention layers, enabling deeper conditioning between both modalities. This model approach also integrates advanced normalization, dropout, and projection layers for better control.

While our final training and evaluations focused on the simpler version of the fusion model due to the training stability and resource constraints, the more complex architecture offered greater modularity. The reasons for this choice will again be discussed in the experiments section.

Furthermore, our implementation supports flexibility. Although the image encoder is frozen during training, the fusion layer and the textual encoder are trained together. This makes the cross-attention weights learn the dataset better over time.

In summary, our fusion architecture uses a cross-attention mechanism to align and integrate visual features with textual tokens. This choice is motivated by its overall clarity, success in similar architectures, and compatibility with transformer-based encoders. By attending to image features for each word in the input question, this module produces context-aware and grounded language representations that make accurate answer predictions.

Large Language Model (LLM)

In our Visual Language Model (VLM), the large language model (LLM) is a crucial block. It is the generative backbone responsible for interpreting the fused multimodal representations and generating meaningful and contextual natural language responses. The role and the structure of the LLM are critical because they convert the aligned vision-language features into meaningful text.

We used a pretrained transformer-based language model in our architecture. This model was not trained from scratch, as training a large language model (LLM) from scratch requires intensive computational resources, vast datasets, and significant financial investment. Consequently, only huge companies with big GPU clusters and financial funding are capable of training large language models from scratch. Thus, instead, we finetuned the pretrained model during our training stage to teach the model the VQA tasks. This pretrained model provides a starting point for our VLM model.

We do not use the entire LLM as a closed module. Instead, we extract its intermediate layers, particularly the input and decoder blocks, which are integrated into our overall model. The textual encoder, which was described in an earlier section, starts by creating embeddings from tokenized inputs. These embeddings are then improved through multimodal fusion.

The final text output is generated by passing vision-aware embeddings through the decoder layers of the LLM. This process results in a series of logits corresponding to the vocabulary of the LLM. These logits show the model's predictions for the next word at each step, which are then turned into natural language responses using methods like greedy decoding or sampling. The overall design of our language model block can be seen in Figure 3.15.

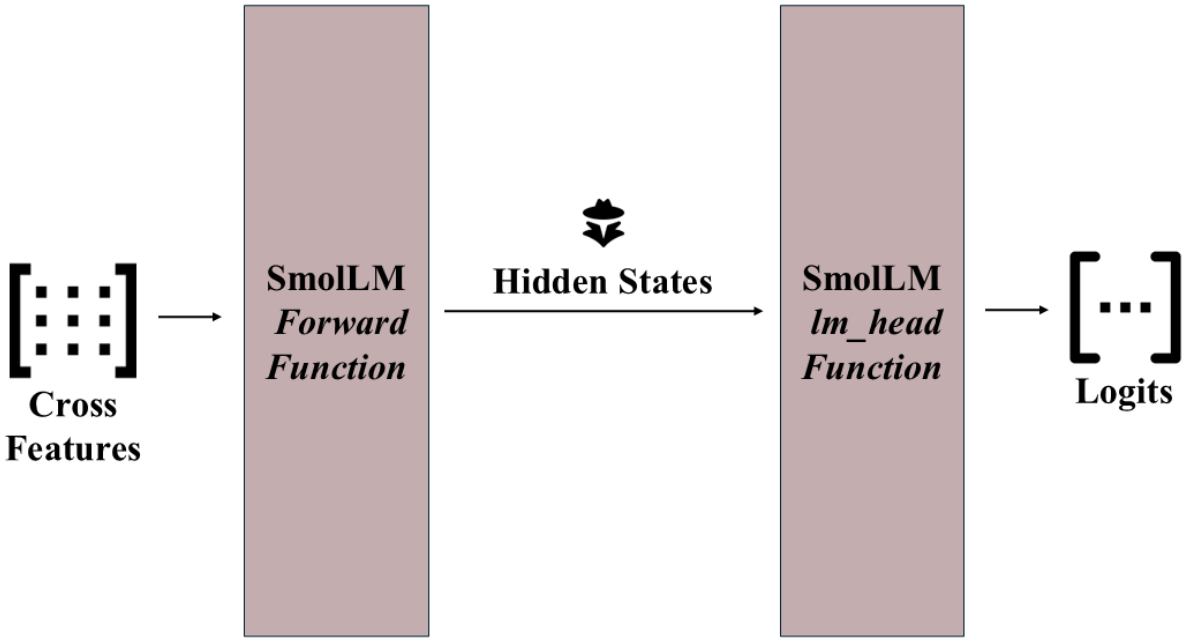

Figure 3.15 General design of the language model block

We intentionally chose a relatively small-scale LLM to limit our resource constraints while still allowing for generative capabilities. Although larger models would provide a better result, they would take significantly higher training, which is not feasible in our current setup.

The architecture of the language model itself wasn't changed, but its parameters were fine-tuned during training. The finetuning is very important as it allows the model to learn the distribution of visual question answering (VQA) data and learn how to integrate the vision conditioned embeddings. Training the language model block, rather than keeping it frozen like the vision encoder, enhances the accuracy and the quality of the generated responses.

Although several alternatives were also considered, such as encoder-decoder setups like T5 [37], the current architecture was chosen for its balance of architectural simplicity, open-source availability, and compatibility with our design.

In summary, our modular design not only allows for the LLM component to be replaced or upgraded independently from the rest of the architecture but also provides flexibility for future improvements.

3.2.3 Implementation

The implementation of the Visual Question Answering (VQA) model was developed using Python, mainly utilizing the PyTorch deep learning framework along with Hugging Face Transformers for working with pre-trained language models. TorchVision was also used for vision related tasks, such as image preprocessing and model loading. Additionally, the Python Imaging Library (PIL)[14] was used for handling image files such as JPG or PNG. Moreover, the pandas library was used to load and manipulate the dataset annotations in a structured format. Lastly, it is important to note that many other libraries are used for various tasks, including data manipulation, visualization, and utility functions. They all contributed to the efficient development of the VLM model.

Development and the training of the models took place in a hybrid environment. The majority of the model training and the experimentations were conducted on the Google Colab Pro[15], environment, utilizing Nvidia A100 40GB VRAM graphical processing unit (GPU). In addition to this, university computer lab machines and local laptops were used to run the test parts of the models.

For the large language model component of the model, several transformer-based models from Hugging Face were used. After careful analysis and selection, the primary model chosen for this project was HuggingFaceTB/SmolLM2-135M. The reason for this decision was its balance of efficiency and performance. It was the smallest LLM that could suitable with our standards. Additionally, its lightweight architecture allowed for faster training and inference times, which were crucial for us. On the other hand, there were other models that we also used to test and train our architecture, such as HuggingFaceTB/SmolLM2-360M and Qwen/Qwen3-0.6B[16]. These models are relatively larger than the first one; however, they were also considered small compared to state-of-the-art VLM models. The Qwen3-0.6B model was particularly important for us because it is a Mixture of Experts (MoE) architecture. One of our aims for this project was to explore and use MoE architecture. This allowed us to further study how MoE architecture affects the VQA tasks.

[14] https://pypi.org/project/pillow/
[15] https://colab.google/
[16] https://huggingface.co/Qwen/Qwen3-0.6B

To make the input textual questions usable for the model, the tokenization of input should be made. For this, the corresponding tokenizer of each model is used. However, the original tokenizers were not specific to our use case. Thus, a custom tokenizer is designed on top of the existing original tokenizers. small set of task-specific special tokens were introduced, including <*start_sequence*>, <*end_sequence*>, <*question*>, and <*answer*>. These tokens were added to the tokenizer vocabulary and used to structure the textual inputs. Each prompt is designed to build the following format: it starts with <start_sequence>, followed by the question enclosed by the <question> token, then the <answer> token, and during the training phase, concatenates the answer followed by <end_sequence>. This structured prompting format allowed the model to clearly differentiate between the question and answer types and enabled to language model to learn our specific tasks. The prompt template function can be seen in Figure 3.16.

```python
def prompt_template(question, answer=None, training=True):
    encoded_prompt = "<start_sequence>"
    encoded_prompt += f"<question>{question}"
    encoded_prompt += f"<answer>"
    if training:
        encoded_prompt += f"{answer}<end_sequence>"

    return encoded_prompt
```

Figure 3.16 Prompt template function code snippet

In order to use the ViT model in our architecture, only minimal modification is needed as can be seen in Figure 3.17. First of all, the classification head of the model was removed and replaced with an identity layer. For this project, only the intermediate features should be extracted without passing them through the classification layer. It should be noted that this replacement should be needed for other vision encoder models, since most of the vision encoders' main purpose is to do classification. This change in the model also ensured that the visual features are extracted in a suitable format for fusion with the language embeddings. The model was initialized in evaluation mode to freeze the parameters during the training as can be seen in Figure 3.18. While our codebase was written to also support the training of the vision encoder, we chose not to train or fine-tune it for this project due to the time and computational limitations.

```
class ImageEncoderVIT(nn.Module):
    def __init__(self):
        super().__init__()
        self.vit_model =
vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1).to(device)
        self.vit_model.heads = nn.Identity()  # Remove the classification head
        self.transform_func = ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1.transforms()

    def forward(self, x):
        x = self.transform_func(x)
        x = x.unsqueeze(0).to(device)
        x = self.vit_model(x)
        return x
```

Figure 3.17 Image encoder class code snippet

One important note is that the model's forward function allows for switching between using precomputed image features and computing them on the fly. When *train_image_encoder* is disabled, image feature extraction is used with *torch.no_grad()* context to save GPU memory and avoid unnecessary processing through the frozen image encoder. This switching possibility provides flexibility for experimenting with either fixed or trainable vision backbones for future experiments.

```
self.image_encoder = ImageEncoderVIT()
        for param in self.image_encoder.parameters():
            param.requires_grad = self.hp.get('train_image_encoder', False)
```

Figure 3.18 Freezing the image encoder parameters code snippet

To enable multimodal training, a custom dataset classes were created, extending PyTorch's Dataset interface as can be seen in Figure 3.19. These class is responsible for handling the preprocessing and pairing of each image-question-answer tuple into a format suitable for the multimodal model. Each entry consists of a raw image path and a formatted prompt string question, and (if training) the answer. The image is initially loaded using PIL and passed through the ViT-based ImageEncoderVIT module to obtain the visual features. At the same time, the prompt is tokenized using the custom tokenizer, which includes special tags like <question>, <answer>, and <start_sequence>. Tokenized sequences are padded and truncated to a maximum length defined in

the hyperparameters dictionary to ensure consistent input sizes during batch processing. The resulting dataset object is then wrapped in a DataLoader. DataLoader created with shuffling enabled and drop_last=True to ensure all batches are equally sized, which is an important step for efficient GPU parallelization.

```
class LlavaDataset(Dataset):
    def __init__(self, dataframe, tokenizer):
        self.dataframe = dataframe
        self.tokenizer = tokenizer
        self.image_encoder = ImageEncoderVIT()
    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, index):
        data_row = self.dataframe.iloc[index]
        image_path = data_row['image']
        prompt = data_row['prompt']

        image = Image.open(image_path).convert('RGB')
        # extract image features using the image encoder
        image_features = self.image_encoder(image)

        # tokenize the question-answer pair
        tokenized_prompt = self.tokenizer.encode_plus(
            prompt,
            return_tensors='pt',
            padding='max_length',
            max_length=hyperparams['max_length'],
            truncation=True
        )
        # return the image features and tokenized prompt
        return {
            'image_features': image_features,
            'tokenized_prompt': tokenized_prompt
         }
dataset = LlavaDataset(df, tokenizer)
dataloader = DataLoader(dataset, batch_size=hyperparams['batch_size'], shuffle=True,
drop_last=True)
```

Figure 3.19 An example LLAVA dataset class code snippet

The fusion between visual and textual data is handled via CrossAttentionFusion module. Internally, it uses a MultiheadAttention layer from PyTorch. It is set to attend over the projected

visual tokens (as key and value) while the textual embeddings act as the query. This design makes it possible that every token in the question can focus on the relevant visual features during the forward propagation in training. The output is combined with the original text embeddings through a residual connection. Lastly, outputs are going through the layer normalization.

The chosen optimizer is AdamW for most of the model training stages. AdamW combines the Adam optimizer with decoupled weight decay. This choice is particularly well-suited for transformer-based architecture. It offers improved generalization by preventing overfitting. A relatively low learning rate of 5e-5 is used in most of the model training. The low learning rate is chosen because of the fact that stable updates to the model's parameters are important at this stage of the training as can be seen in Figure 3.20.

```
loss_fn = torch.nn.CrossEntropyLoss(ignore_index=tokenizer.pad_token_id)
optimizer = torch.optim.AdamW(
    params=model.parameters(),
    lr=5e-5,
    weight_decay=0.01
)
```

Figure 3.20 Loss function and optimizer definition code snippet

The model is trained across multiple epochs. Each epoch is defined in the hyperparameter dictionary by the name num_epochs. During each epoch, a progress bar tracks the average training loss and prints to the screen. After each epoch, the model's state dict (weights) is saved to local disk in a checkpoint folder.

During inference, the model uses an autoregressive generation loop that replicates the decoding behavior of large language models. The generate function takes an image and a question as input, computes the vision combined embeddings, and begins to generate the response. At each step, it extracts the last token's logits and selects the most probable next token using various sampling methods, then appends it to the input. This process repeats until either the maximum length is reached or an <*end_sequence*> token is produced.

In order to use to model to generate a textual response, various sampling methods are used. These sampling methods that we are going to discuss are important because they significantly affect the quality, diversity, and relevance of the generated text. By selecting the most suitable sampling

techniques, we can control aspects such as randomness and determinism in the output. This is crucial for producing coherent and contextually appropriate responses in Visual Question Answering (VQA) tasks and even the LLM field. We selected three primary sampling techniques. These techniques are greedy decoding, top-k sampling, and nucleus (top-p) sampling.

Greedy Decoding

The greedy decoding method is a relatively simple strategy. It selects the token with the highest probability at each step of generation. At each timestep t, the model selects the token w_t with the highest conditional probability as can be seen in Eq. 3.1.

$$w_t = argmax\, P(w \mid w_1, w_2, \dots, w_{t-1}) \tag{3.1}$$

This method is deterministic and fast in its nature, making it useful for debugging, testing reproducibility, or generating answers that need to strictly follow training patterns. This can lead to a highly confident response. However, it can also lead to repetitive or less diverse outputs, especially for longer sequences, because it ignores lower-probability tokens that might be a semantically better choice. It should be noted that the language itself is not just about always selecting the highest probability word; it involves creativity, context, and sometimes ambiguity. For example, consider a sentence, “the cat sat on the…”. Greedy decoding might always choose the “mat” as the highest possibility for the coming word. However, other valid and creative answers could include "couch," "windowsill," or "roof," each providing a different context or meaning to the sentence. Another example in the context of Visual Question Answering (VQA) might be that consider an image showing a busy street scene with various objects such as cars, pedestrians, and buildings. If the question asked is, "What is the main object in the image?", greedy decoding might always select "car" if it's the most frequently occurring object in the training data. However, other valid answers, such as "pedestrian," "building," or "traffic light" might be a better answer for that image and question. Thus, there is a need for another sample method to capture the full range of possibilities.

Top-k Sampling

Top-k Sampling improves over greedy decoding by using stochasticity. Instead of always selecting the highest probability token, the model restricts the candidate set to the top k most

probable tokens at each timestep and samples from this subset. Formally, define the top-k subset as can be seen in Eq. 3.2.

$$V_k = \{w \in V \mid P(w \mid w_{<t}) \; is \; among \; the \; top \; k \; values\} \tag{3.2}$$

The next token w_t is then sampled from the normalized distribution over V_k. This method introduces randomness and can produce more diverse outputs than greedy decoding. Moreover, it avoids extremely low-probability tokens from being chosen as the next token in the sequence. In practice, values like $k = 40 \; and \; k = 50$ are common, although for our setting and model, we conducted that the best value for k is between $5 \; and \; 7$. This choice is highly dependent on the model's vocabulary and training data.

Top-p Sampling (Nucleus Sampling)

Top-p Sampling improves the top-k method by adapting the size of the candidate set dynamically based on the cumulative probability. Instead of a fixed number of top tokens, it considers the smallest set $V_P \subseteq V$ as can be seen in Eq. 3.3.

$$\sum_{w \in V_P} P(w \mid w_t \;) \geq P \tag{3.3}$$

Typically, p is used between $0.8 \; and \; 0.95$. Top-p sampling is more flexible than top-k, making it suitable for this project.

All three sampling strategies were tested and tuned, and they are all used to see what they generate for different types of images and questions. A causal language modeling approach is used during training. This means that the input tokens are shifted by one position to form the prediction targets. More precisely, given a token sequence $x = [x_1, x_2, \dots, x_n]$, the model is training to predict $x_{x+1}$from x_t. Consequently, the target labels are created by slicing $input\_ids[:, 1:]$ while aligning the logits by slicing $logits[:, : -1, :]$. Both tensors are then flattened to compute the loss value over the whole batch.

3.2.4 Evaluation of VLM

Evaluating Visual-Language Models, specifically the ones that use generative sampling, is a complex and unresolved problem in the AI community. Unlike classification models that produce only discrete outputs, easily verifiable against their labels. However, generative models produce open-ended natural language responses. These responses can vary in form, length, and style, yet still be equally valid or even superior in terms of the quality of the usage of the language.

To this day, rating the output of generative models remains an open challenge because of several reasons:

- **Multiple Valid Answers**: A single visual question may have multiple responses. For example, a question like *"What is happening in the image?"* can be answered correctly as "A man is playing tennis" or "An athlete is hitting a ball with a racket". These answers are both language-wise true, but traditional metrics like BLEU or ROUGE would penalize them due to word mismatches.
- **High Sensitivity to Wording:** Metrics like BLEU, ROUGE-L, and METEOR are N-gram-based metrics. They are not very good at scoring the paraphrases. Even small differences in word order or synonym choice can cause large drops in score.
- **Lack of Ground Truth**: For many VQA questions, there is no single standard answer. This makes the supervised evaluation difficult.
- **Partially Correct Answers:** Some answers may be partially correct or contain small errors.
- **Language Generation Quality:** Beyond just factual correctness of the answer, generated responses should also be fluent. Traditional metrics do not evaluate the ability of linguistic coherence.
- **Hard to Debug:** When a model fails, analyzing why it failed is hard. It might misunderstand the image, or misread the question, or maybe the fusion mechanism failed. This makes evaluation and debugging much harder than in unimodal models.

To solve these problems, we developed a hybrid evaluation strategy combining automatic and human assessments.

Automatic Metrics

We calculated 4 metrics. They are BLEU, ROUGE, METEOR, and BERTScore [38]. While BLEU, ROUGE, and METEOR are fast and easy to compute, they are not suitable for evaluating open-ended generative VQA. Thus, BERTScore is chosen as our main metric. It uses embeddings from the BERT model to assess the similarities between generated and reference answers.

Human Evaluation:

To further solve the problems of automatic metrics, we conduct user evaluations. In this process, participants were asked to rate generated answers on a 10-point scale. Additionally, participants were shown paired answers from two different models: one from our proposed VLM model and the other from the competitive SmolVLM model family. Users were asked to choose the answer they preferred for each question. We manually prepared 6 different questions from different categories for 5 images, which adds up to 30 questions in total. These questions are from various types of visual understanding tasks. This made the evaluation beyond standard benchmarks. Images were selected from the COCO test set. All model outputs were anonymized and randomized during the comparisons.

Even after this evaluation design, evaluating generative visual question answering models remains a fundamentally difficult task. Even BERTScore, does not fully score reasoning or partial correctness. At the end, our hybrid approach balances automatic evaluation and human evaluation.

3.2.5 Limitations of Visual Understanding

Although our model shows very promising results in multimodal reasoning, it has several limitations in its visual understanding capabilities, particularly when compared to large-scale state-of-the-art Visual Question Answering (VQA) models. These limitations mainly arise from multiple including model size, training duration, dataset diversity, and generalization constraints.

One of the most important limitations is the model's difficulty in answering questions that require precise, fine-grained visual understanding. The model specifically performs poorly on question types such as:

- **"How many..." questions:** Counting objects requires structural awareness and attention over multiple visual tokens. Our model is limited by its relatively basic image

encoder and lack of specialized counting training often inaccurately estimates object quantities, particularly when objects overlap or vary in size.

- **"What color..." questions:** Identifying the object colors involves low level understanding of the visual embeddings. This ability may not be well kept in the high-level visual embeddings (especially those from models like ViT). Since our architecture focuses more on scene content than pixel level details, color related questions often lead to incorrect answers.
- **"Is it..." or binary questions**: Yes/no questions are a particularly tricky type of question because they require precise alignment between the visual information and textual data. Even small errors in attention can result in the wrong binary prediction. Moreover, this type of question is widely open to hallucination.

These limitations are particularly due to the model's modest scale, both in terms of trainable parameters and the total number of training iterations. Because of the hardware and time limitations, the model has not been trained for many epochs and has not been shown to a large dataset distribution as more advanced VQA systems. As a result of this, some of these questions might lead to a falsely generated result.

Despite its weaknesses on these question types, the model shows notable strengths in scene-level understanding. It generally performs better in high level questions such as:

- "What is in the image?"
- "What sport is being played?"
- "What objects are present?"

In these types of questions, the model effectively recognizes important patterns and object types. Moreover, it produces a meaningful description. This suggests that the cross-attention fusion mechanism is effective in scene understanding tasks.

Another important limitation to be noted is the restricted visual domain. Although the model is theoretically designed to accept all types of images, its performance is significantly better on images from the COCO dataset. This is not coincidental since three of the datasets used during training (VQA v2, LLAVA, and GQA) are heavily based on COCO images. This causes the model to become familiar with those types of images. As a result, when it is asked about an out-of-domain image, such as Sefaköy Central Mosque, the model's behavior becomes unpredictable. It may

generate unrelated answers and may fail to recognize some objects. While theoretically all images could be used and get good results, it is strongly recommended to limit inference inputs to COCO test or validation images to reduce unpredictability and maximize answer quality.

3.3 Stage 3: Web Application Design and Architecture

In this part of the project, a responsive and user-friendly web application is served. The aim is to provide an interface where individuals can upload images, ask questions about those images, and receive AI-generated answers from the multimodal VQA model in real-time. In addition to the main property of the application, users can register and view their old messages after logging into the application again.

It is important to indicate that this is not an exact real-time chatbot but rather a single-turn interaction system. It means that users submit a question along with an image and receive a single answer in return. This means the application does not support continuous or multi-turn dialogue over the same visual context. This design was chosen to simplify model interaction and minimize server-side memory usage. Also, it is requested to give the project's main focus on VQA inference rather than dialogue modeling.

3.3.1 Purpose and Use Case

As we discussed above, the main purpose of the web application is to deliver an interactive demonstration for the VQA system developed as part of the graduation project and make the web application as user-friendly as possible. By allowing individuals to use a complex multimodal model through a simple interface, the system shows that it is practical and user-friendly, making advanced AI more accessible to everyone. When it is considered about the use case, it is straightforward but technically intensive: a user registers to the application, logs in to the application, uploads an image, chooses a model, asks a question related to the image, receives a coherent answer generated by the VQA model, and views his/her old messages.

3.3.2 Feature List

- **User Registration**: Users can create an account by entering a username, email, and password. The system saves this information for future login.
- **User Login**: Users can log in by entering their registered e-mail and password. If the information matches, they access the application.
- **Image Upload**: Users can upload images directly from their devices using a simple drag-and-drop area or a traditional file selector. This image is ready as a visual input for the VQA model.
- **Text Input for Questions**: A text box is provided for individuals to enter a question about the uploaded image. The system is designed to understand and process different types of questions.
- **Model Selection**: Users can choose between different models if they wish before submitting a query. This allows comparison between models.
- **Real-Time Response**: After the submission, the application sends the image and question to the backend, which later sends them to the VQA model. The model processes the input and returns a response that is then shown to the user.
- **Dynamic Loading Indicator**: While the model is generating an answer, a kind of spinner is displayed to inform the individual that the system is working. This helps manage expectations and improve the interaction flow.
- **Popup Display of Answers**: The model's response is shown in a popup window, clearly separating the result from other UI elements and drawing attention to the model's answer.
- **Error Detection**: The system checks for common problems like missing inputs or server-side errors. If an error happens, the user is notified through a message that helps avoid confusion.
- **Input Validation**: The frontend prevents the user from sending a request if the image or question is missing or if the image file format is not supported. In that way, it is ensured that only valid data is sent to the backend.
- **View Past Messages**: Logged-in users can view their previous questions and the corresponding answers. This feature helps users reflect on past interactions and compare model outputs.

3.3.3 Frontend Frameworks Used

The front end was developed using React.js[17], a popular JavaScript[18] library known for its component-based architecture and reactive user interface capabilities. In addition, TypeScript[19] was used for static typing, which improves code maintainability and reduces runtime errors. With these technologies, a modern, maintainable, responsive, and user-friendly frontend that integrates with the backend and model layers was constructed.

3.3.4 UI Design

The user interface (UI) was designed with clarity and simplicity. It ensures a great experience for both technical and non-technical users. The design utilizes card components to separate sections (e.g., image upload, text input, and result display). Minimalistic and modern styling with modern colors improves readability, while feedback animations like button hover effects improve interactivity. Accessibility was also considered for the UI design, with readable fonts and semantic HTML elements for screen reader support.

In Figure 3.21, the registration page is shown with 4 text input boxes including username, email, and password. Also, a button is there to send the request to the backend services, and an anchor to send users to the login page if they already have an account.

[17] https://react.dev/
[18] https://www.javascript.com/
[19] https://www.typescriptlang.org/

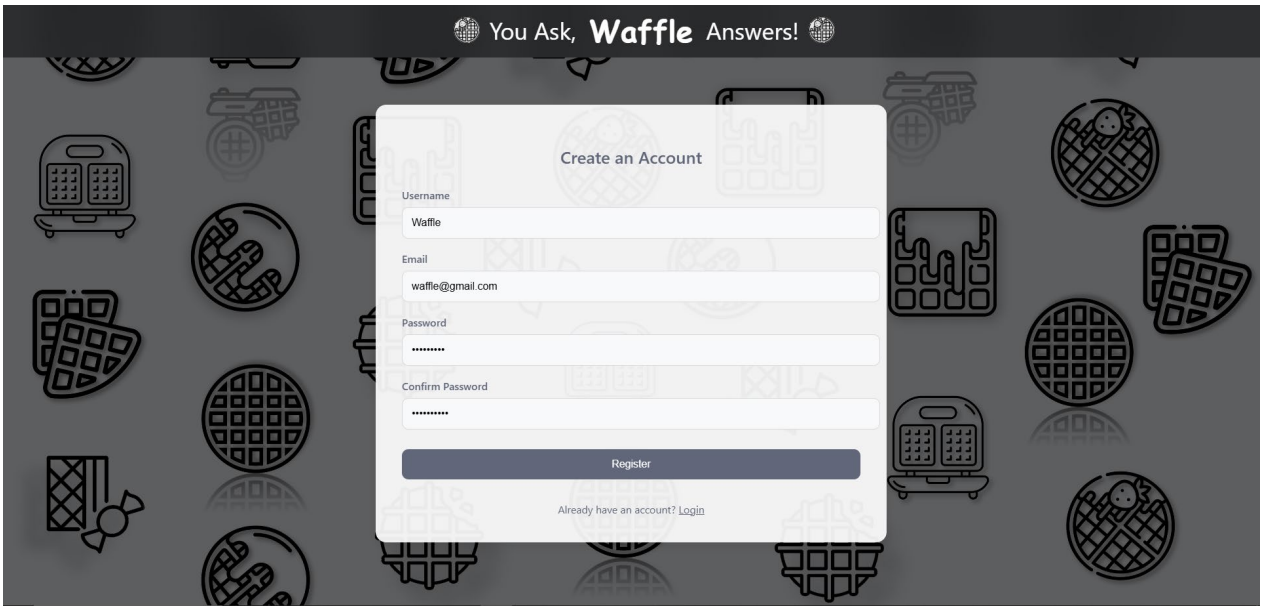

Figure 3.21 Registration page

After the registration page, users can log into their account, as can be seen in Figure 3.22. There are 2 input text fields, including e-mail and passwords. The button below the text fields allows users to send the data to the backend services easily and log into their account. Also, there is an anchor at the bottom of the white box, allowing users to create an account by sending them to the registration page.

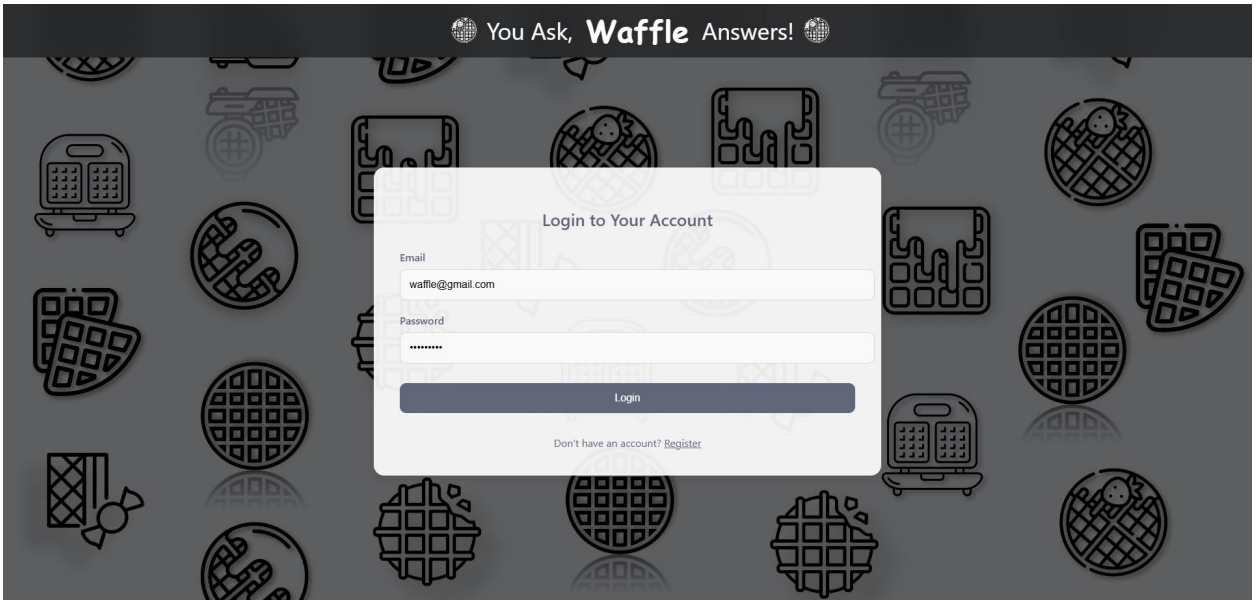

Figure 3.22 Login page

As can be seen in Figure 3.23, there is a text field including the question. Below the text field, there are 3 buttons. One of them includes a camera icon, which provides users with upload images. The other button allows users to open the images they've just uploaded, as can be seen in Figure 3.24. Finally, the button with an arrow sends the query and the image to the backend services.

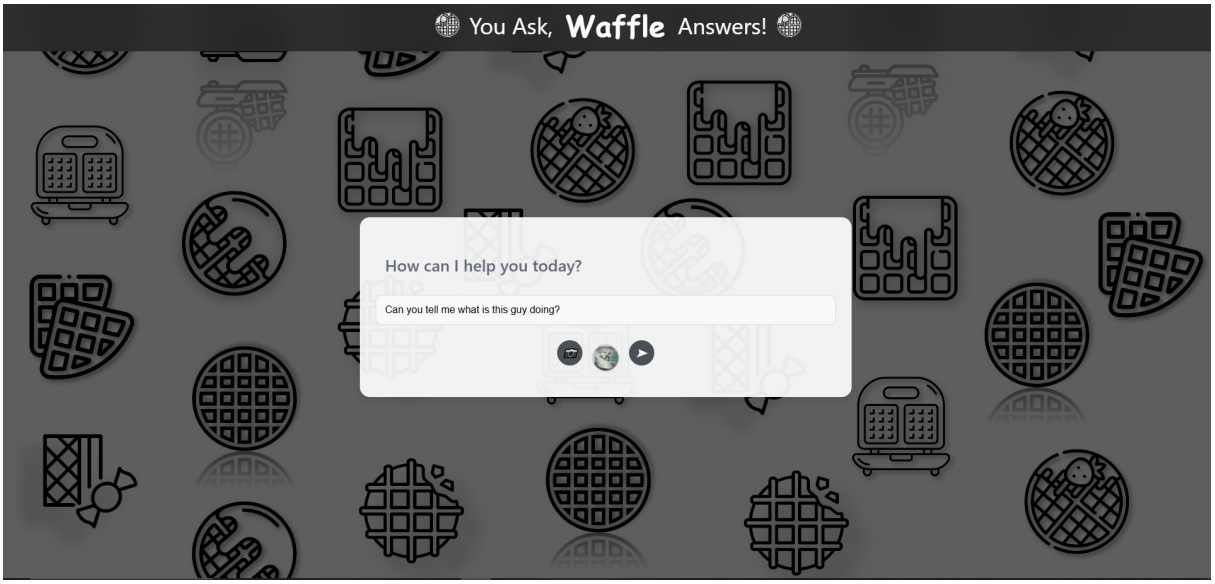

Figure 3.23 Main page

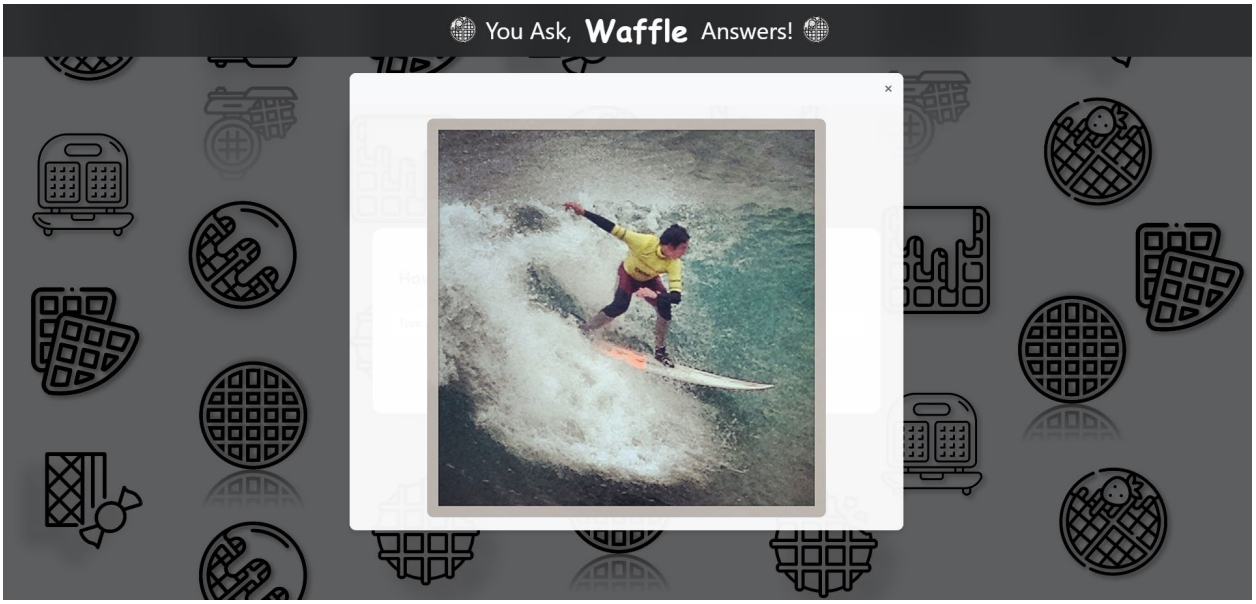

Figure 3.24 Displaying the uploaded image

While waiting for the answer, a blurred screen with an animated icon is served to users, as can be seen in Figure 3.25. In that way, users ensure that their query is being processed right now.

Figure 3.25 Waiting screen

In Figure 3.26, users can see the image they've uploaded, the question they've asked, and the model's answer. Below the answer, there is a button that helps users to ask another question.

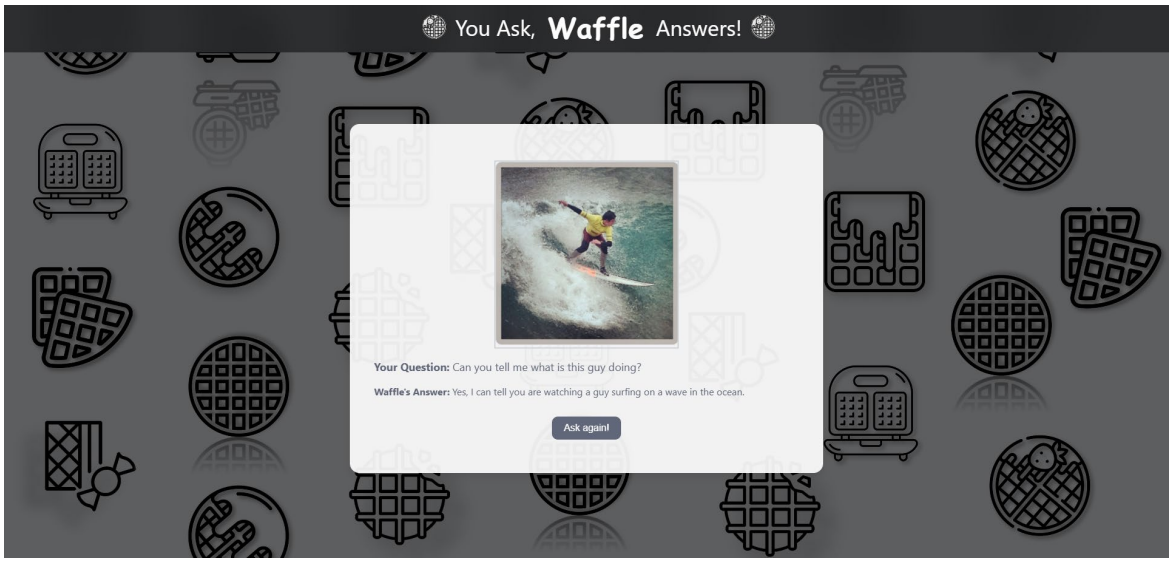

Figure 3.26 Result page

Finally, users can view their search history as can be seen in Figure 3.27. There are small white boxes in a big transparent white box displaying the image, question, and answer. Thanks to

the buttons on the left and right sides, users can slide to the left and right to find the requested query. In addition, users can go back to ask for more with the button at the bottom of the box.

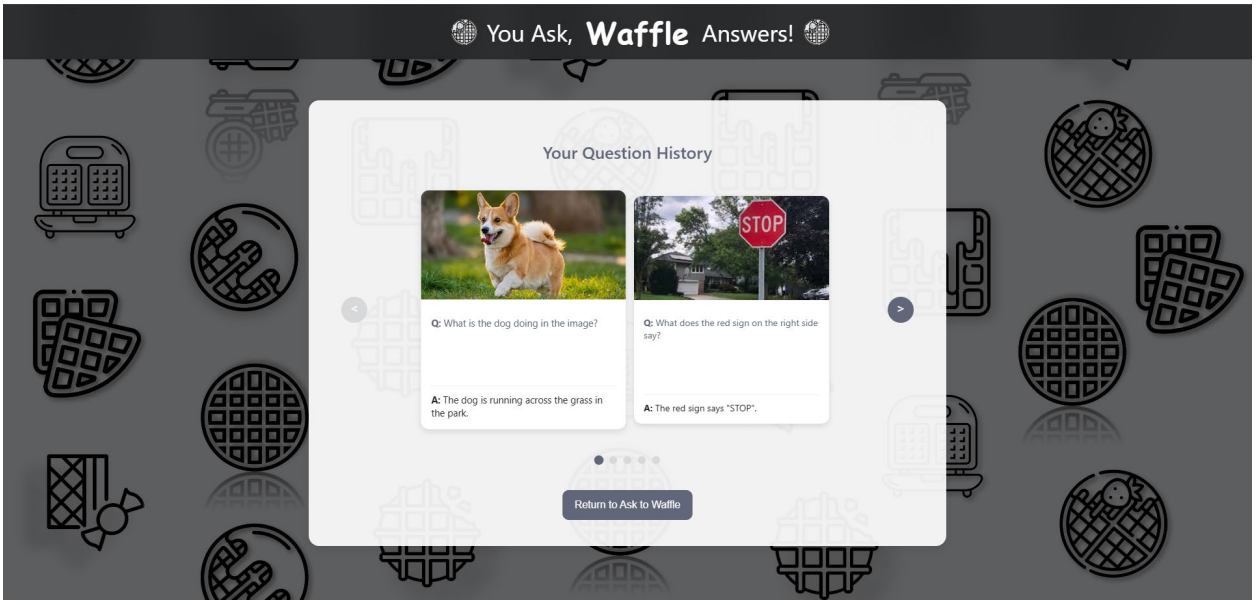

Figure 3.27 History page

3.3.5 Frontend Implementation

The implementation of the frontend utilizes a modular and scalable architecture. The code is structured to improve clarity, maintainability, and separation of concerns. Within src directory, the project is divided into meaningful subfolders such as apis, components, pages, config, and locales, as can be seen in Figure 3.28.

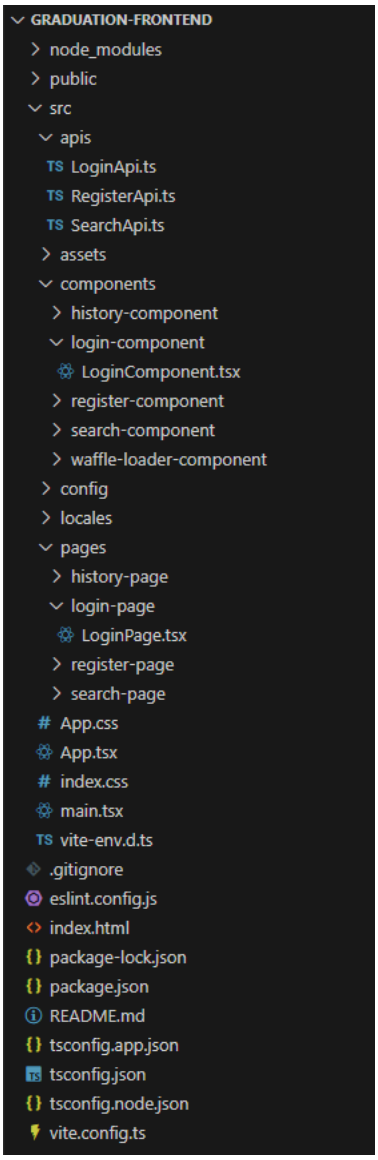

Figure 3.28 Front end structure

Each section of the application has its own page located under the pages directory. For example, login-page, register-page, search-page, and history-page, as can be seen in Figure 3.28. These page directories are responsible for rendering the primary layout. Inside each page, the relevant view is shown by integrating modular components from the corresponding folders in components.

The components directory is further broken down into subfolders like login-component, register-component, search-component, history-component, and waffle-loader-component, as can be seen in Figure 3.28. These subfolders include React functional components responsible for

rendering specific UI elements and encapsulating their logic. With this approach, it is ensured that each part of the UI can be developed, tested, and maintained independently.

API interactions are placed in the apis folder, as can be seen in Figure 3.28. TypeScript files LoginApi.ts, RegisterApi.ts, and SearchApi.ts are utilized to define functions for communicating with the backend services. Asynchronous operations are handled using the native fetch() API, and responses are utilized to drive state transitions within the application, such as toggling loading indicators, displaying result popups, or handling errors.

State management is handled using React hooks such as useState and useEffect. These hooks ensure dynamic rendering and response to user interactions so that the interface is responsive and interactive. Styling is managed through global and modular stylesheets (App.css, index.css), and the application is built using Vite. Finally, this structured approach to frontend development improves scalability, enhances code reusability, and supports efficient team collaboration.

3.3.6 Backend Technologies Used

The backend of the application was built using Spring Boot[20], a widely used Java framework that simplifies web application development and provides built-in support for REST APIs, dependency injection, and data handling.

Communication between the frontend and backend was established using a RESTful API, which handles HTTP requests to send image and text data. The backend serves endpoints like /ask, /login,/register and etc. For instance, the endpoint /ask accepts a multipart/form-data request containing the uploaded image and user-entered question. When the backend service receives the data, it processes the input and forwards it to the Python-based model layer, then sends the generated text answer back to the frontend.

For data storage, MongoDB[21] was used as the database technology thanks to its flexibility and efficiency in handling JSON-like document structures, which aligns well with storing image metadata, user messages, and model outputs. Since MongoDB is a NoSQL database, it does not use JDBC for data access. Instead of that, the backend uses Spring Data MongoDB, which is more suitable for working with MongoDB collections through repositories and document mapping.

[20] https://spring.io/projects/spring-boot
[21] https://www.mongodb.com/

These technologies provide a scalable and developer-friendly backend environment that can serve both the current application needs and future improvements.

3.3.7 Backend Structure

The backend was designed following the Layered Architecture (N-Tier Architecture) pattern [39]. This architecture separates the backend into different layers, each with a specific property, as can be seen in Figure 3.29. This modular approach improves code readability, maintainability, and testability.

- **Presentation Layer**: Handles incoming HTTP requests from the frontend (via REST endpoints), validates inputs, and returns HTTP responses.
- **Application Layer (Service Layer):** Handles the primary business logic. It processes the input data, calls the model service, and manages flow control.
- **Data Access Layer**: Interacts with the MongoDB database and performs operations like saving user queries or image metadata.
- **Model Layer**: The model layer is where the VQA model is executed. This structure separates the machine learning part from the main backend. Therefore, the machine learning computations are done in a Python program, not in a Java backend.

This clean separation ensures that if there is a change in one layer, it does not affect the others much, and each part can be developed and tested independently.

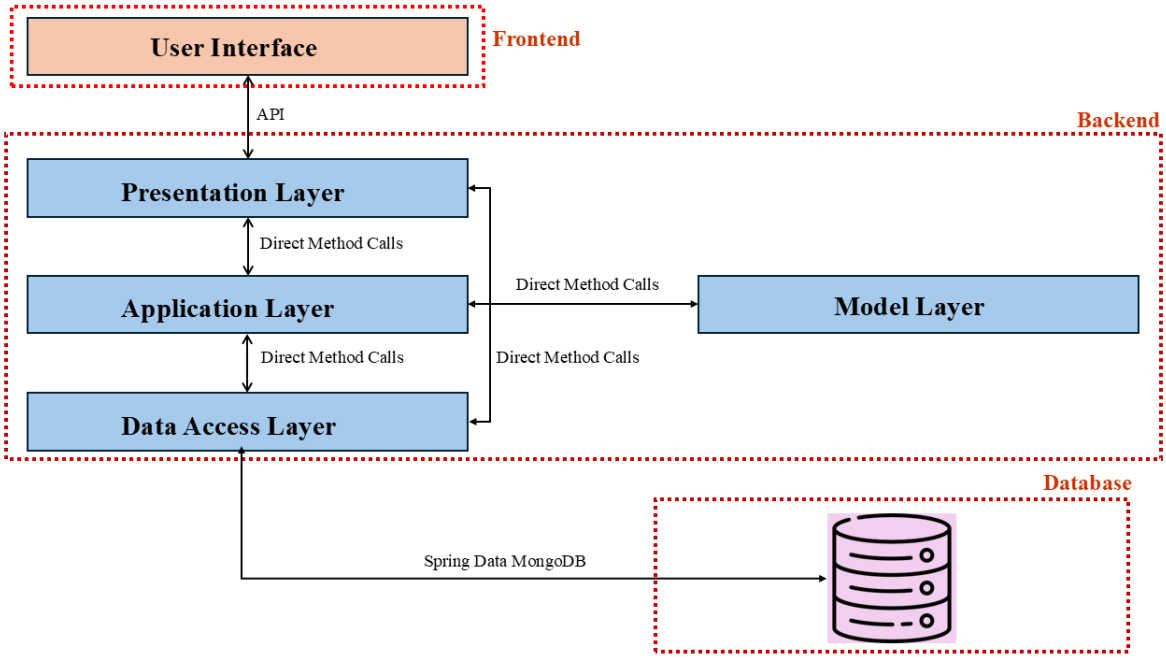

Figure 3.29 Layered architecture

3.3.8 Error Handling and Security

In this web application, basic error handling and security measures are implemented to ensure the application remains stable and protected against common issues. The backend checks all inputs to be sure that there are no problems before processing them. For example, it verifies that both an image and a question are provided and that the image is in a supported format.

If something goes wrong, like a failed connection to the model layer or an internal server error, the system responds with clear error messages and proper HTTP status codes, such as 400 for Bad Request or 500 for Internal Server Error. This feature helps users understand what occurred and allows developers to find and fix problems easily.

On the security side, the web-based application applies basic protections such as blocking unsupported file types and enabling CORS (Cross-Origin Resource Sharing) [40] to control which frontends can send requests to the backend services. Access to the database is secured by enabling authentication, which requires users to log in to the application with proper credentials.

3.3.9 Backend Implementation

The backend was implemented using Spring Boot, which provides a strong foundation for building RESTful web services. The backend has endpoints that receive data and then forward it to the Python-based VQA model, and return the result to the front end. This logic was divided into multiple layers based on the Layered Architecture: Presentation Layer, Application Layer, Data Access Layer, and Model Layer, as we discussed in the previous sections.

In Figure 3.30, the layers can be seen clearly. The presentation package is the Presentation Layer, which handles HTTP requests and responses. The service package is the Application Layer, where the main business logic is implemented. This layer processes the received input, communicates with the VQA model, and prepares the output to be sent back. The database package is the Data Access Layer, which manages all interactions with the MongoDB database. It includes components that handle data storage, retrieval, and queries. Lastly, the model folder is the Model Layer where the VQA model is executed. This layered structure enhances the maintainability, scalability, and clarity of the backend system.

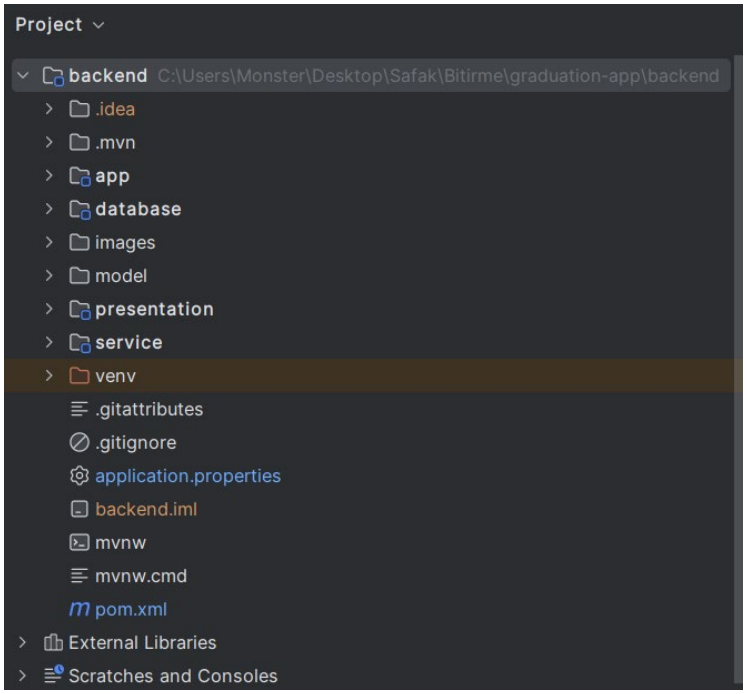

Figure 3.30. Layered architecture implementation

The term "breakdown" or "decomposition" in English is important to understand the significance of this layered approach. This breakdown is not only achieved through separating the application into layers like presentation, application, data access, and model, but also through dividing each layer into meaningful subfolders. For instance, subfolders like handler, implementation, mapper, and type help divide responsibilities further within a layer. With this structure, each component is isolated and easy to locate. Consequently, the code becomes cleaner, more modular, and much easier to maintain, debug, and scale over time.

In each package, there are several subfolders to construct this decomposition. They are used for a modular and clean backend structure. In Figure 3.31, there is the architecture of the presentation layer. The Handler sub-package includes interfaces, which are used to process incoming HTTP requests, and the implementations of the interfaces are done in the implementation sub-package. This sub-package carries out the business logic needed to deal with each request. Therefore, it can be stated that it is the main sub-package. The mapper sub-package includes classes that convert data between different formats. For example, from requests objects to response objects or database entities and vice versa. In that way, it ensures a clean separation between layers. There is the last sub-package named type that includes the structure of data exchanged between the client and the server. The architecture is nearly the same for all other layers, but there are some differences. Service and presentation sub-packages' subfolders are exactly the same, but the database layer has entity instead of type and repository instead of handler. Finally, instead of putting the model layer into another service that can be reached via HTTP request, this layer is integrated within the project structure as a separate Python environment so that the Java backend communicates with the model locally, and it enhances performance by minimizing network overhead.

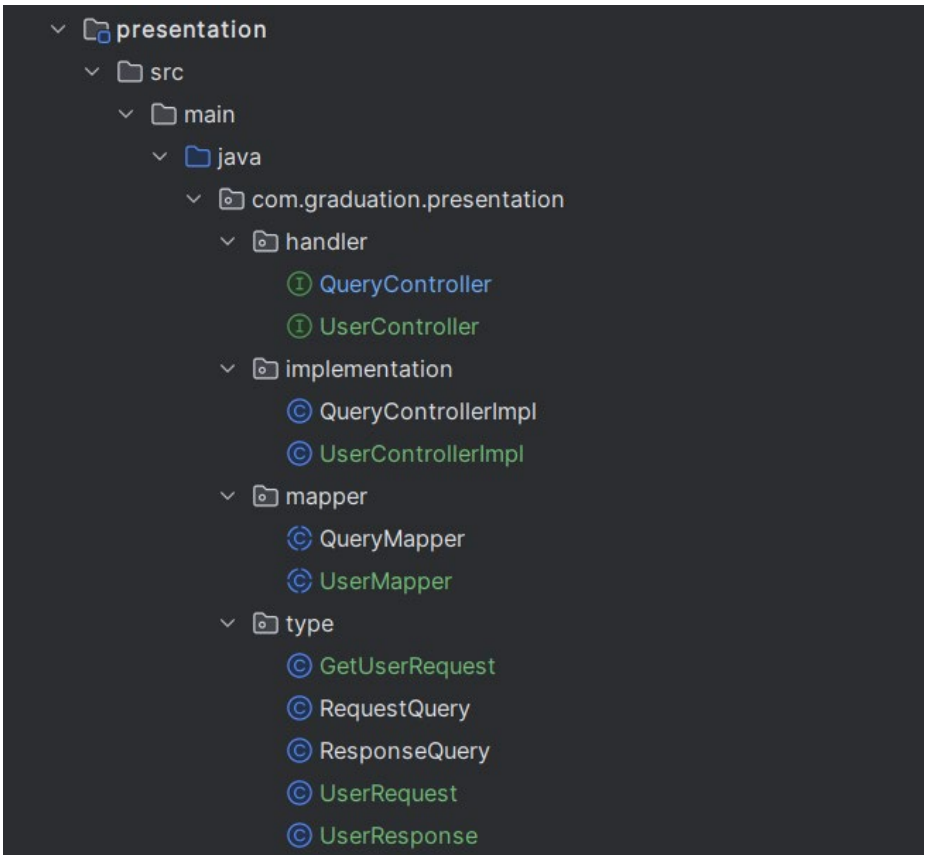

Figure 3.31 Breakdown implementation

3.3.10 Scalability

Scalability is a significant consideration for any system. As mentioned in the project proposal, horizontal and vertical scaling are two main approaches to address growing requests [41]. Horizontal scaling involves distributing the application across multiple machines, which increases performance but requires load balancing mechanisms, either hardware or software-based, that can be costly and complex to implement as can be seen in Figure 3.32. On the other hand, vertical scaling involves upgrading the current system's hardware resources, such as CPU, RAM, or GPU, to deal with more load as can be seen in Figure 3.33. Because of the limited time, budget, and the scope of the graduation project, horizontal scaling was not used in this project, but the web-based application was designed to run on a single personal computer. The architecture and implementation were optimized for vertical scaling on a single machine. This results in the application and machine learning models running smoothly in a development setting. If the system

were to be developed further for real-world use, horizontal scaling could be considered as a next step to improve reliability and handle higher traffic.

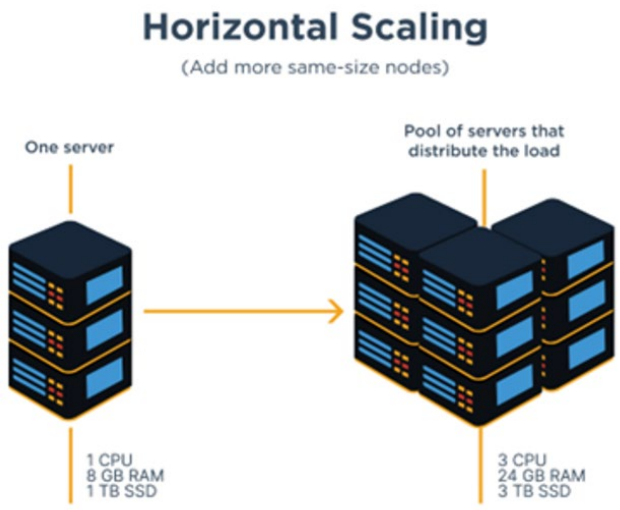

Figure 3.32 Horizontal Scaling [42]

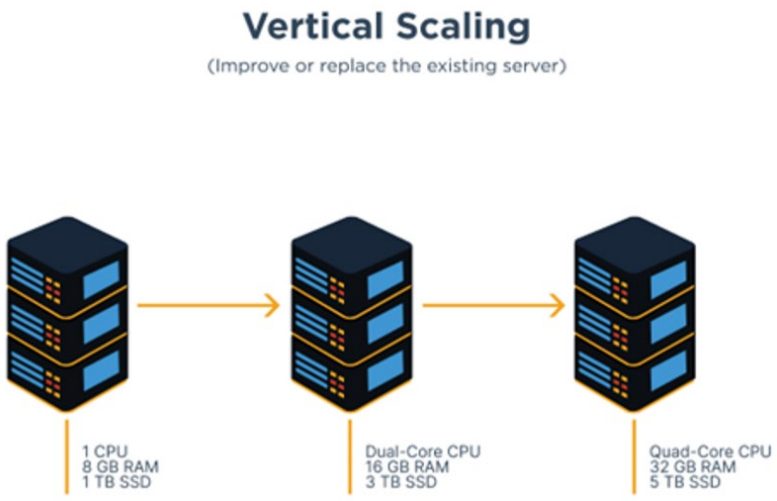

Figure 3.33 Vertical Scaling [42]

4 EXPERIMENTS AND RESULTS

Stage 1:

In this section, the experimental process and findings from Stage 1 will be talked about, focusing on the finetuning of a language model on text only datasets. The aim of this stage was to get the best language model for Stage 2 multimodal training.

Training Environment

The training is done both on our local consumer grade GPUs and Colab Pro Nvidia A100 40GB VRAM GPU. All trainings are done with mixed precision (fp16) to reduce memory usage and improve training speed. The average time per epoch was approximately 1.5 hours across datasets, which changes on token length and batch size. We used a fixed learning rate of 5e-5 and the AdamW optimizer without extensive hyperparameter tuning.

Each dataset was independently finetuned on the base model for 10 full epochs. This resulted in three distinct LLMs. Moreover, we also trained the models using cross-dataset method. This cross-dataset method was done to test the benefits of sequential fine-tuning. For example, the SQuAD-trained model was later further trained on Dolly. This created a hybrid model designed to balance both instructional understanding and factual answering. In total, six fine-tuned models were produced during this stage:

- Dolly-only
- SQuAD-only
- TriviaQA-only
- SQuAD → Dolly
- Dolly → SQuAD
- TriviaQA → Dolly

The average total time taken was for each model was approximately 90 hours. Since there are 3 distinct models to train (SmolLM2-135M, SmolLM2-360M, and Qwen2.5-0.5B) the total the time required for the training all the models approximately 270 hours. This approximation does not include the test trainings done for precision settings.

Stage 2:

In this section, the experimental process and findings from Stage 2 will be talked about, which focuses on the full Visual Language Model (VLM) training for the Visual Question Answering (VQA) tasks. In this stage the vision and language features were fused using our custom cross-attention architecture.

Due to the significant resource demands for training even small sized multimodal models, most of our training in this stage is done on the Colab Pro Nvidia A100 40GB VRAM GPU.

As discussed in other sections, in Stage 1 we finetuned our language models using only text based question answer datasets. We assume that it would help with Stage 2 in multimodal training. However, during early experiments in Stage 2, we noticed something surprising: using the finetuned base model didn't help much when it is used in the multimodal learning. Because of this surprising finding, we changed our approach and decided to start Stage 2 from scratch using the base models only.

This choice made our training system simpler. More importantly, it gave us an insight that when working with both text and images, it's often better to train the model from the base model in a multimodal way, rather than using text-only finetuned model. The learning process changes a lot when vision is added, and Stage 1 tuning didn't transfer as well as we expected.

We run many training sessions during Stage 2 using different datasets and model settings. While some were full trainings, many of them were test trainings that still took significant time.

For the VQA v2 dataset, we trained two main versions of our model. The only difference between them was the number of attention heads, 8 heads and 12 heads. Both models were trained for 4 epochs, each took about 8 hours in float16 precision mode. We chose float16 to save memory and speed up training. The same epochs in float32 would have taken nearly 16 hours. Thus, we did not train on float32.

On the GQA dataset, we trained three versions of our model using 100K, 300K, and 500K samples from the full dataset. These training sessions helped us understand how model performance changes with more data, but the training details differ from running to running (e.g., the number of epochs wasn't always consistent)

Lastly, the LLaVA dataset was used to train another set of models for 6 epochs, with each epoch taking roughly 3 hours to finish.

4.1 Automatic Metrics

In Section 3.1.2.4, we mentioned about four automatic evaluation metrics that were used to calculate the performance of our multimodal model. These are BLEU, ROUGE-L, METEOR, and BERTScore. BLEU focuses on precision through n-gram overlap, ROUGE-L emphasizes recall via longest common subsequence matching, METEOR uses precision, and BERTScore utilizes embeddings to compute similarity between the predicted and actual answers.

For our first set of experiments, we evaluated the model's performance across different training epochs. In these epochs, the number of heads was kept at 8. Also, this model was trained on the VQA v2 dataset. The aim of this analysis was to demonstrate how the model's ability to produce accurate answers developed with additional training.

The performance metrics for each epoch are summarized in Table 4.1. From the table, we can observe a consistent improvement in all metrics as the number of epochs increases. To summarize, the progression across epochs shows that continued training helps the model to use the language and answer the questions correctly on the VQA task. The relatively delayed improvement in BLEU states that the model may initially focus more on semantic similarity than strict n-gram precision. This is common in generative models for tasks involving open-ended language.

Table 4.1 Epochs Over Different Automatic Evaluation Metrics

	BLEU	**ROUGE-L**	**METEOR**	**BERTScore**
Epoch 1	0.0000	0.4361	0.2258	0.9766
Epoch 2	0.0000	0.4839	0.2518	0.9781
Epoch 3	0.0000	0.5489	0.3100	0.9837
Epoch 4	**0.0181**	**0.5757**	**0.3272**	**0.9859**

Moreover, as shown in Figure 4.1, the most significant increase happens at epoch 3. Especially in the METEOR, ROUGE-L, and BERTScore metrics. By looking at Figure 4.1, we confirm that extended training leads to meaningful gains across all evaluation metrics. On the other hand, each epoch needed a significant amount of time to finish, making it impractical to maintain

training beyond the fourth epoch within the scope of our available resources due to hardware limitations.

As it was discussed in section 3.2.4, BERTScore is the best choice for our graduation project as it fits well with the aims of multimodal generation, accounts for semantic correctness, and handles the diverse, free-form nature of answers in VQA tasks. Since it focuses on meaning rather than exact word matching, it is useful to evaluate models that generate language from combinations of visual and textual information.

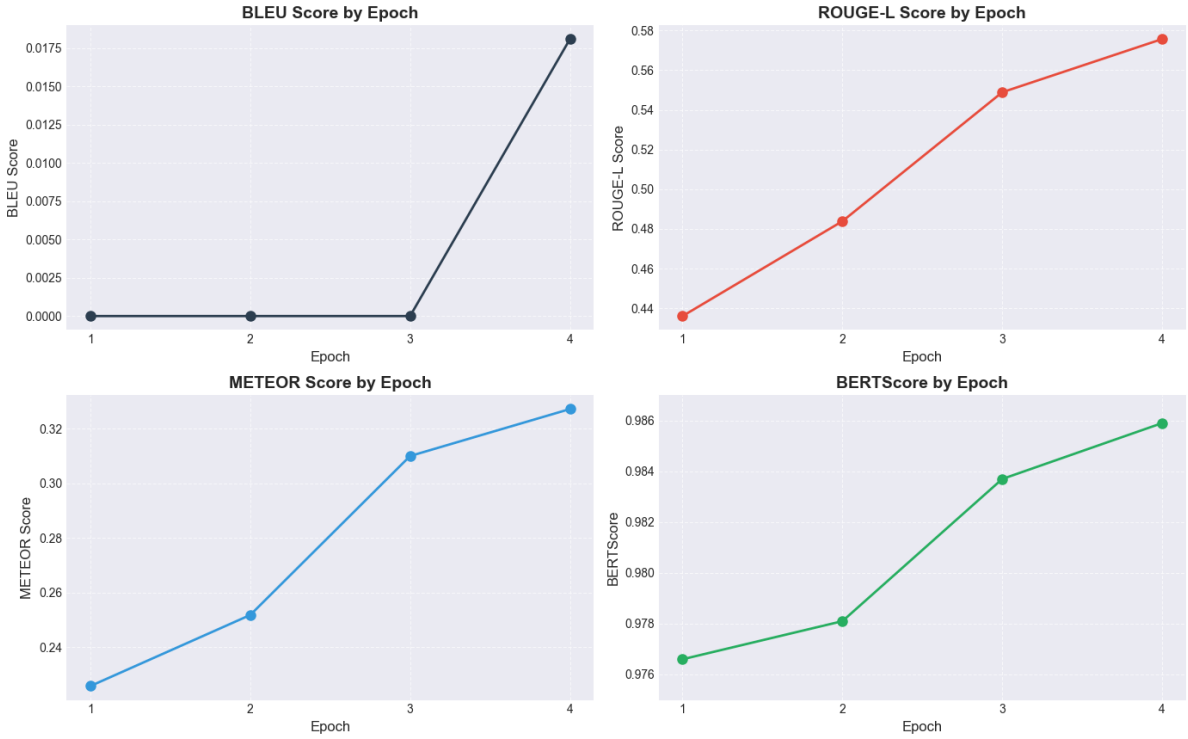

Figure 4.1 VQA v2 model performance over epochs (Number of Head = 8)

For our second set of experiments, we evaluated the model's performance across different training epochs with the number of attention heads set to 12 as can be seen in Table 4.2. Also, this model was trained on the VQA v2 dataset, too. It can be stated that more epochs increase the correctness, too, like the number of heads set to 8. We can still examine a consistent improvement in all metrics as the number of epochs increases, but the difference comes from the values. We can see that the values are higher when the number of heads set to 8 rather than 12, as can be seen in Figure 4.2.

Table 4.2 Epochs Over Different Automatic Evaluation Metrics (Number of Heads = 12)

	BLEU	ROUGE-L	METEOR	BERTScore
Epoch 1	0.0000	0.4110	0.2144	0.9759
Epoch 2	0.0000	0.47164	0.2457	0.9782
Epoch 3	0.1703	0.5349	0.2970	0.9812
Epoch 4	**0.1965**	**0.5673**	**0.3005**	**0.9833**

In general, the purpose of this analysis was to examine how increasing the number of training epochs affects the model's capability to generate accurate responses under different attention settings. As can be seen in Figure 4.2, using the number of heads set to 8 allows the model to focus on the most relevant parts of the input during training. Although it might be supposed that a larger number of heads offers better results, it seems that in our specific architecture and data scale, 12 heads may introduce unnecessary complexity or overfitting.

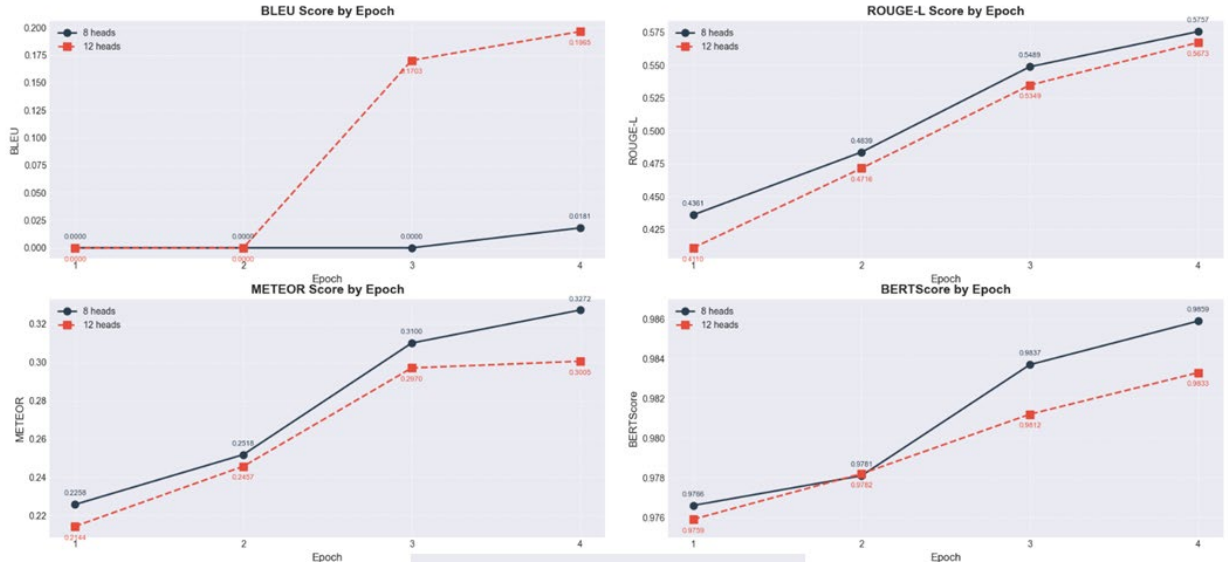

Figure 4.2 VQA v2 model performance over epochs (Number of Heads = 8 vs Number of Heads 12)

The comparison between the best-performing epochs for both attention head configurations can be seen in Table 4.3. When the number of attention heads is set to 8, the model reaches better scores across all evaluation metrics, as discussed in the previous paragraph.

Table 4.3 Best-Performing Epochs Over Different Automatic Evaluation Metrics

	BLEU	**ROUGE-L**	**METEOR**	**BERTScore**
Best model from number of heads is 8	0.0181	0.5757	0.3272	0.9859
Best model from number of heads is 12	0.1965	0.5673	0.3005	0.9833

It is decided to continue the experiments with the model in epoch 4 and having the number of heads set to 8. Different datasets were also used to train the model in order to observe which one fits the best with our goals. VQA v2, GQA, and LLAVA datasets were used, as can be seen in Figure 4.3. First, the model was trained on only the VQA v2 dataset. Then, the model was individually trained on GQA and LLaVA datasets, too. Moreover, two combined datasets were used. The model was trained on the VQA v2 dataset first and then further trained separately on the GQA and LLAVA datasets.

As can be seen in Figure 4.3, the model achieved the best results when the model was trained with only the VQA v2 dataset. The reason why the BERTscore is higher for VQA v2 compared to LLAVA is that VQA v2 consists of concise questions and answers that are easier for the model to learn and predict accurately. On the other hand, LLAVA includes more natural, longer, and complex sentences, making it more challenging for the model to achieve equally high scores. Another reason might be that the evaluation was made using the VQA v2 dataset. However, it can be concluded from the figure that combined datasets reduce the performance and GQA dataset is not good enough to use it in our graduation project.

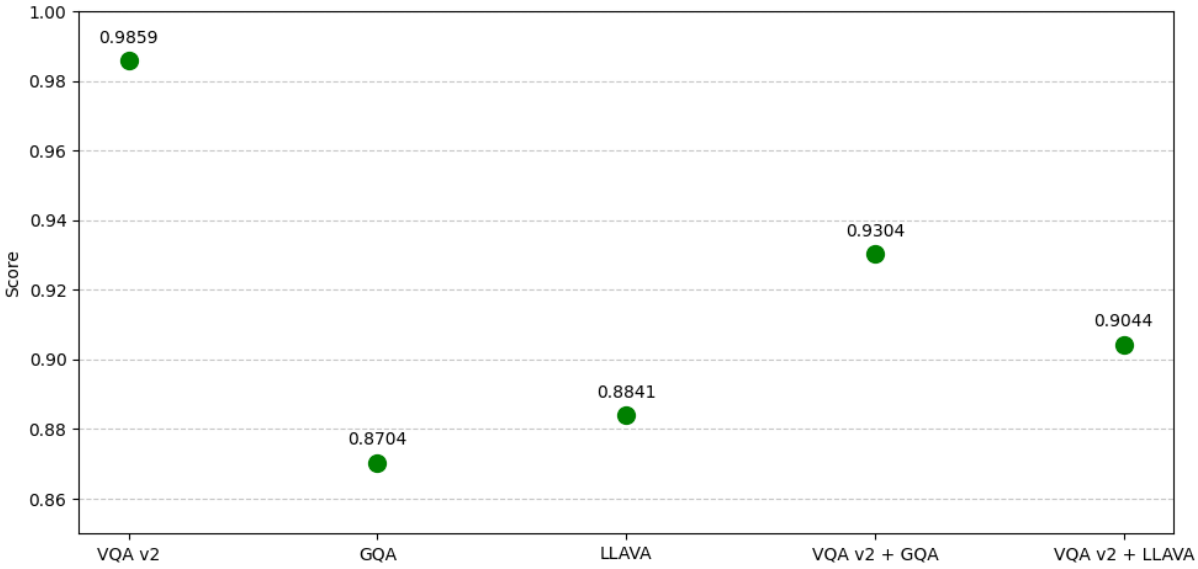

Figure 4.3 Model Training Comparison on VQA v2, GQA, and LLAVA datasets

In addition to the dataset comparisons, the model's performance was evaluated on different subsets of the GQA dataset with varying sizes: 100K, 300K, and 500K samples. As can be seen in Figure 4.4, increasing the number of training samples from 100K to 500K leads to improved accuracy, with the highest accuracy of 0.8704 achieved using the full 500K subset. It shows that larger dataset sizes positively affect the model's ability to learn and generalize within the GQA domain. However, we could not continue with larger datasets again because of the hardware limitations mentioned in section 3.2.5.

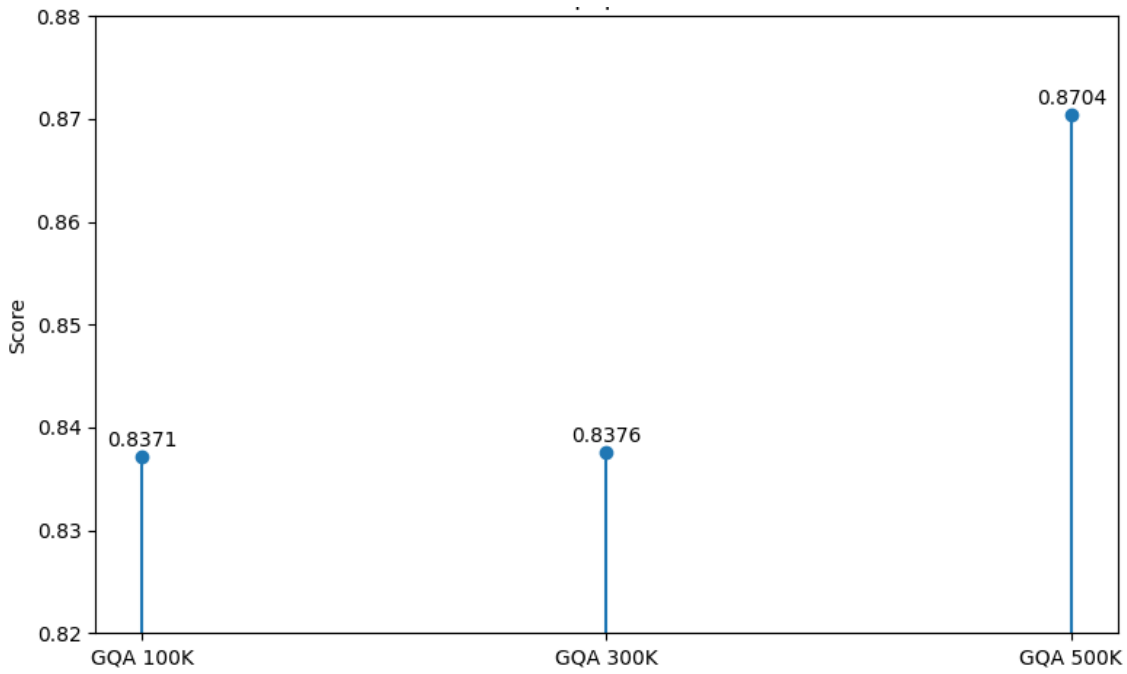

Figure 4.4 BertScore accuracy on different-sized GQA subsets

Even though the best results come from VQA v2, we used LLAVA for our graduation project since it has more natural language and longer sentences, which fit well with our graduation project expectations. As can be seen in Figure 4.5, the model's performance on the LLAVA dataset improved continuously over epochs. Starting from an accuracy of 0.8220 at Epoch 1, the model achieved 0.8841 by Epoch 6, demonstrating consistent learning and adaptation of the dataset. As can be guessed, we could not maintain training for more epochs because of the time and hardware limitations again. On the other hand, we believe that our model is still performing well given the constraints. Despite these limitations, the final results obtained on the LLAVA dataset show that our model effectively learns from natural structures, making it a strong fit for our project's real-world, multimodal question answering objectives.

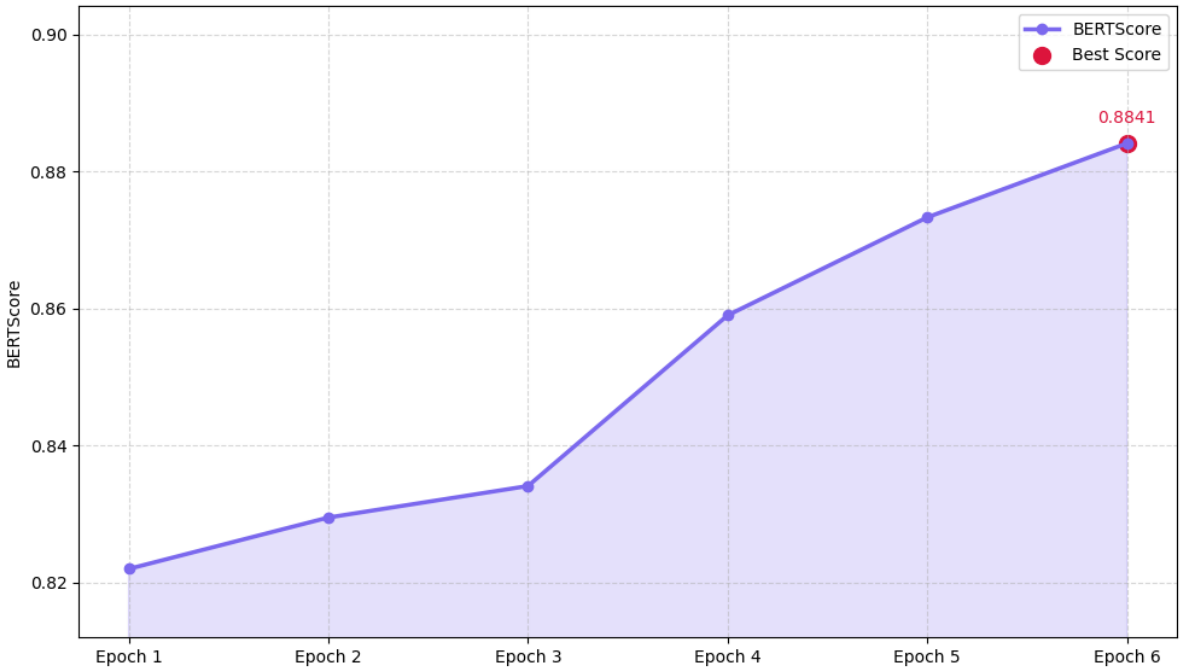

Figure 4.5 BertScore accuracy on the LLAVA dataset across training epochs

4.2 User Experience

A user study was conducted with 30 participants. One of the goals was to observe the comparative performance of our proposed model and a baseline model (SmolVLM-256M-Instruct) across different question types. Moreover, another aim was to obtain a score that can help us to determine the model's performance. We wanted to emphasize that the main aim was not to determine which model was the best because it is not appropriate to claim such a thing due to the reasons mentioned in section 3.2.4, but the primary purpose of conducting a user study is to understand which model performs better in which question categories.

Also, the demographic information of the participants was not systematically collected because we wanted to collect information about general impressions rather than statistically grounded conclusions while conducting a user study. On the other hand, it can be stated that a diverse group of individuals from 22 to 60 years old, including both males and females, and representing different occupational and academic backgrounds participated in the study.

There were 5 images and 6 questions from different categories related to those images. The categories are:

- Counting
- Object Recognition
- Scene / General Understanding
- Advanced Reasoning
- Attribute-based Questions
- Subjective Questions

There is an example image from the study as can be seen in Figure 4.6. Moreover, there are related questions that can be seen in Table 4.4.

Figure 4.6 An example image of a jet fighter flying in the sky from the study

Table 4.4 Related Questions of Figure 35 from the Study

Counting	How many vehicles are there in the image?
Object Recognition	What is the vehicle in the image?
Scene/General Understanding	What is the vehicle in the image doing?
Advanced Reasoning	What kind of professionals are trained to use this vehicle?
Attribute-based	What color is the vehicle in the image?
Subjective	How would you feel seeing this jet flying over your city?

It was requested by each participant to give a point to each answer from 0 to 10. 0 indicates a completely irrelevant or not sensible response, while 10 represents a highly accurate and contextually appropriate answer. Also, each participant commented on model outputs without knowing which model generated which response to keep bias at the minimum level.

The results demonstrated that our model was preferred by 25 participants whereas SmolVLM-256M-Instruct was preferred by 5 participants among 30 participants. Therefore, it can be stated that approximately 83.3% of the participants favored our model over SmolVLM-256M-Instruct. This preference can be shown in Figure 4.7, which presents the distribution of model preferences in the form of a pie chart.

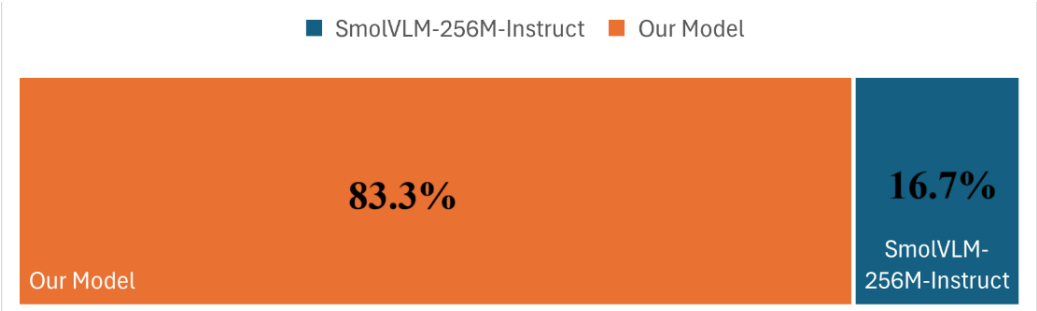

Figure 4.7 Preference distribution between models

In addition to the overall model preference, the average scores given by each participant for all 30 responses (5 images × 6 question types) was evaluated too. A bar chart was created to visualize this data, where each bar represents the average score given by a single participant for

each model as can be seen in Figure 4.8. Our model is shown in orange color, while SmolVLM-256M-Instruct is represented in blue. This figure provides a more detailed view of individual-level preferences and emphasizes the consistency of higher average scores for our model across the participant group.

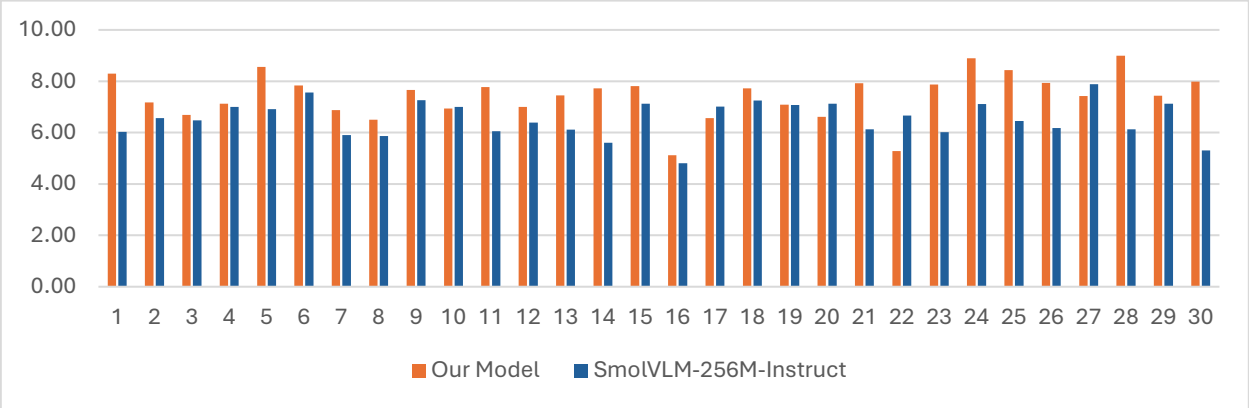

Figure 4.8 Average participant scores for each model given by participants

Figure 4.9 also demonstrates that in the majority of cases, our model received higher ratings compared to SmolVLM-256M-Instruct. In detail, the mean score for our model was 7.42, while SmolVLM-256M-Instruct received a lower mean of 6.54. This difference shows a consistent preference for our model across participants. If the edge cases are considered, the lowest rating for our model was 5.12, while the lowest for SmolVLM was 4.81. In addition, the highest rating for our model was 8.99, while the lowest for SmolVLM was 7.89.

The presence of outliers such as the lowest score of 5.12 for Our Model and highest score of 7.89 for SmolVLM emphasizes important edge cases, where individual preferences or specific question-image combinations may influence perception. Furthermore, since we use sampling in order to generate answers from our model, some answers would be different from the others due to the fact that sampling is stochastic in nature. These exceptions provide useful insights for future work, especially in understanding model errors and making enhancements. They highlight possible weaknesses in certain cases or differences in how users perceive the quality of the answers.

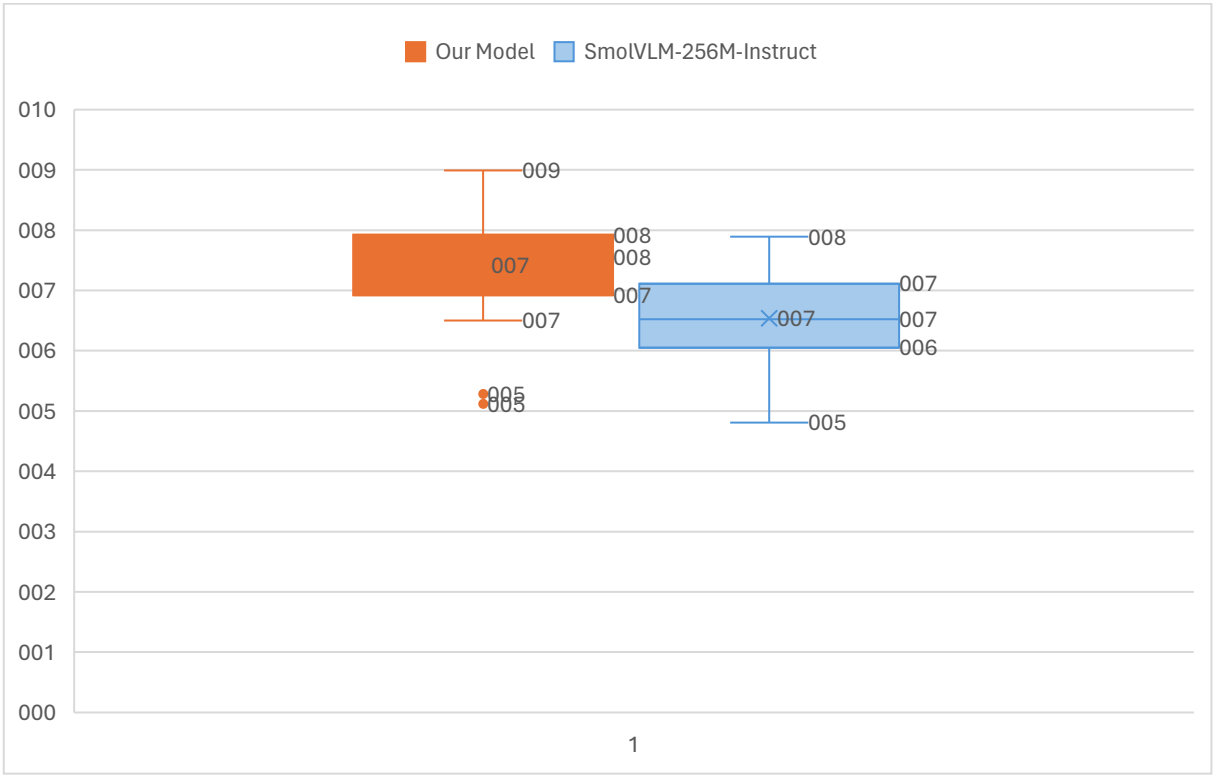

Figure 4.9 Box plot comparison of participant average scores for our model and SmolVLM-256M-Instruct

In order to understand the goal of the user study, a radar chart was generated to visualize their performance across six different question categories. As can be seen in Figure 4.10, our model outperforms SmolVLM-256M-Instruct in four out of six categories which are Object Recognition, Scene / General Understanding, Advanced Reasoning, and Subjective Questions. On the other hand, SmolVLM-256M-Instruct performs better in Counting and Attribute-based Questions.

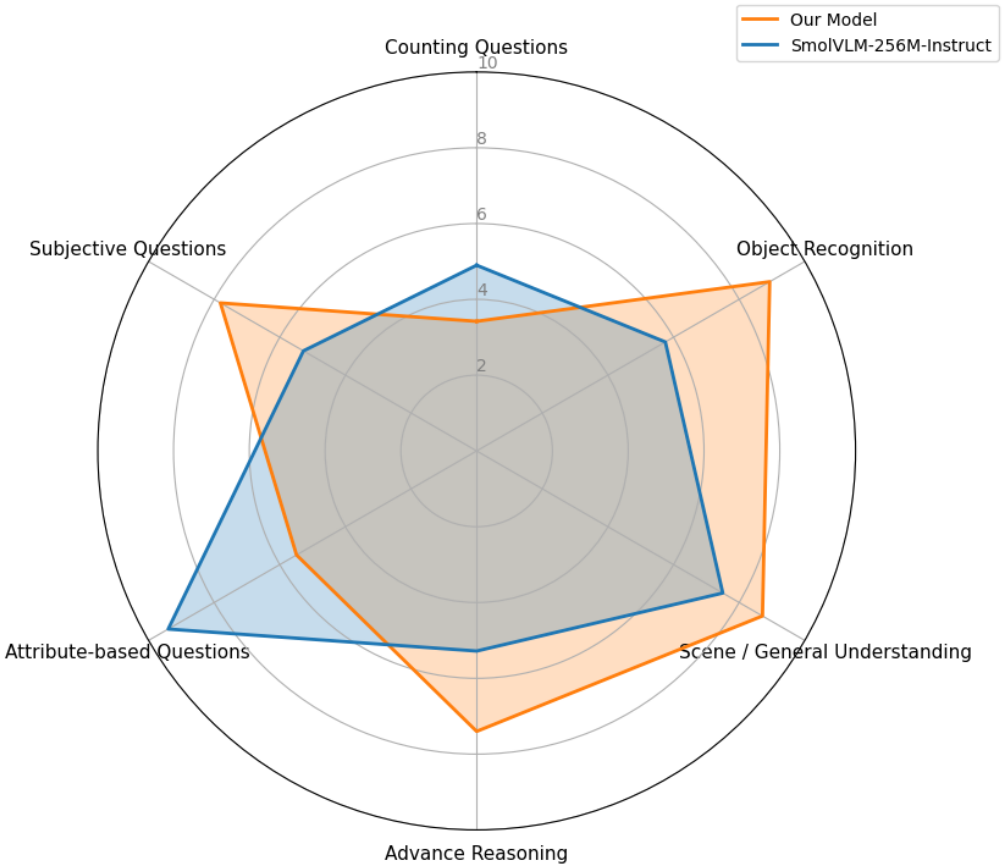

Figure 4.10 Radar chart comparing the performance of the models across six question categories

These results indicate that our model is better for handling complex reasoning, interpretive tasks, and general scene understanding, while SmolVLM-256M-Instruct model performs better on tasks requiring structured or detail-specific analysis. We believe the reason for this is related to several factors, including:

- The size of the dataset
- The variety of the data
- The availability of computational resources
- Limited time for training
- The fact that the other model was trained by a major company (Hugging Face) which has access to significantly greater funding and infrastructure.

This last analysis gives a better understanding of our model's strengths and limitations.

5 DISCUSSIONS

Despite the successes achieved in model performance and system integration, some limitations were encountered throughout the project. One of the main challenges was the high computational cost which is required while training larger multimodal models, especially those including complex architectures such as Mixture of Experts (MoE). This cost is mostly related to high GPU VRAM capacity, increased memory bandwidth, powerful GPU cores, and overall computational resources required for training and inference.

MoE models require substantial GPU resources, including large amounts of VRAM and processing power. Because of the limited availability of such hardware, we were unable to fully utilize MoE's potential or scale the model to the size of state-of-the-art systems like GPT-4. Therefore, we focused on smaller, compact, and efficient model components that could be trained within our resource constraints.

Although we purchased access to Google Cloud Console[22] to obtain more powerful hardware, we were limited by the free usage constraints. We could not continue to use it after the free trial because it was very expensive. For instance, using an NVIDIA A100 40GB GPU on Google Cloud costs approximately $4.05 per hour. We also subscribed to Google Colab Pro, which offers enhanced computer availability. On the other hand, even with this subscription, we encountered limitations such as Colab stating that the session times out after 24 hours. However, during our tests, even with Colab Pro, sessions lasted a maximum of 8 hours before disconnecting, which limited continuous training.

[22] https://console.cloud.google.com/

6 CONCLUSION

In this project, the area of Visual Question Answering (VQA) was covered by the team members. This area focuses on integrating computer vision and natural language processing to enable models to understand and reason about both images and textual queries. The main purpose of this project was to design, train, and evaluate a multimodal AI system capable of answering questions based on image content. While implementing the system architecture, a pretrained vision encoder, a language model, and a fusion mechanism were utilized to effectively combine visual and textual features. Also, a Mixture of Experts (MoE) architecture was examined to improve the model's adaptability and performance. However, it was observed that MoE-based models require significant computational resources to achieve optimal results, making it challenging to fully benefit from their potential within our available hardware constraints. Finally, a web-based application with a layered software architecture was developed, allowing users to interact with the model in a user-friendly environment by uploading images and asking related questions.

The key contributions of this project include the development of a multimodal VQA model, where a modular and compact architecture was implemented using pretrained components to enhance performance across varied datasets. Throughout the project, comprehensive training and experimentation were conducted. Various training configurations were tested, including changes in attention heads, different combinations of datasets, and varying training epochs to better understand the model's behavior and generalization ability. The model was trained and compared on individual and combined datasets such as COCO, GQA, and LLAVA, which provided valuable insights into the strengths and weaknesses of each dataset in the context of VQA tasks. Additionally, the project explored the integration of a Mixture of Experts (MoE) architecture to increase the model's flexibility and specialization. Although MoE showed potential for improved performance, the high computational requirements limited the scale and depth of our experiments. To make the system accessible and interactive, a user-friendly web application was developed using React for the frontend, Spring Boot for the backend, and a Python-based model server. Finally, the system was built following a layered software architecture, separating concerns across the presentation, application, data access, and model layers. This modular structure improved both maintainability and scalability, aligning with best practices in software engineering and modern AI system deployment.

REFERENCES

[1] Agrawal *et al.*, "VQA: Visual question answering," *International Journal of Computer Vision*, vol. 123, no. 1, pp. 4–31, Nov. 2016. doi:10.1007/s11263-016-0966-6

[2] N. D. Huynh, M. R. Bouadjenek, S. Aryal, I. Razzak, and H. Hacid, "Visual question answering: from early developments to recent advances -- a survey," 2025, arXiv. doi: 10.48550/ARXIV.2501.03939.

[3] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, 'Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering', in Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[4] D. A. Hudson and C. D. Manning, 'GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering', in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 6693–6702.

[5] H. Liu, C. Li, Q. Wu, and Y. J. Lee, 'Visual Instruction Tuning', in *Advances in Neural Information Processing Systems*, 2023, vol. 36, pp. 34892–34916.

[6] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, 'Multimodal Machine Learning: A Survey and Taxonomy', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2019.

[7] A. Dosovitskiy et al., 'An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale', *ArXiv*, vol. abs/2010.11929, 2020.

[8] T. Brown et al., 'Language Models are Few-Shot Learners', in Advances in Neural Information Processing Systems, 2020, vol. 33, pp. 1877–1901.

[9] H. Touvron *et al.*, 'LLaMA: Open and Efficient Foundation Language Models', *CoRR*, vol. abs/2302.13971, 2023.

[10] J. Lu, D. Batra, D. Parikh, and S. Lee, 'ViLBERT: pretraining task-agnostic visiolinguistic representations for vision-and-language tasks', in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA: Curran Associates Inc., 2019.

[11] D. Zhang, R. Nayak, and M. A. Bashar, "Exploring fusion strategies in deep learning models for multi-modal classification," *Communications in Computer and Information Science*, pp. 102–117, 2021. doi:10.1007/978-981-16-8531-6_8

[12] A. Singh, V. Goswami, and D. Parikh, ‘Are we pretraining it right? Digging deeper into visio-linguistic pretraining’, *ArXiv*, vol. abs/2004.08744, 2020.

[13] Tamay Besiroglu, S. A. Bergerson, A. Michael, L. Heim, Xueyun Luo, and N. Thompson, “The Compute Divide in Machine Learning: A Threat to Academic Contribution and Scrutiny?,” Unpublished, 2024, doi: 10.13140/RG.2.2.14682.56004.

[14] J.-B. Alayrac *et al.*, ‘Flamingo: a visual language model for few-shot learning’, in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, New Orleans, LA, USA, 2022.

[15] X. Chen *et al.*, ‘PaLI-3 Vision Language Models: Smaller, Faster, Stronger’, *ArXiv*, vol. abs/2310.09199, 2023.

[16] K. He, X. Zhang, S. Ren, and J. Sun, ‘Deep Residual Learning for Image Recognition’, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[17] S. Hochreiter and J. Schmidhuber, ‘Long Short-Term Memory’, *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] A. Vaswani et al., ‘Attention is All you Need’, in Advances in Neural Information Processing Systems, 2017, vol. 30.

[19] P. Anderson *et al.*, ‘Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering’, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6077–6086.

[20] J.-H. Kim, J. Jun, and B.-T. Zhang, ‘Bilinear attention networks’, in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, Montréal, Canada, 2018, pp. 1571–1581.

[21] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, ‘Adaptive Mixtures of Local Experts’, *Neural Computation*, vol. 3, no. 1, pp. 79–87, 03 1991.

[22] N. Shazeer *et al.*, ‘Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer’, in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*, 2017.

[23] D. Lepikhin *et al.*, ‘GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding’, in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.

[24] W. Fedus, B. Zoph, and N. Shazeer, 'Switch transformers: scaling to trillion parameter models with simple and efficient sparsity', *J. Mach. Learn. Res.*, vol. 23, no. 1, Jan. 2022.

[25] H. Tan and M. Bansal, "LXMERT: Learning Cross-Modality Encoder Representations from Transformers," *Empirical Methods in Natural Language Processing*, pp. 5099–5110, Aug. 2019, doi: 10.18653/V1/D19-1514.

[26] L. H. Li, M. Yatskar, D. Yin, C.-J. Hsieh, and K.-W. Chang, 'VisualBERT: A Simple and Performant Baseline for Vision and Language', *ArXiv*, vol. abs/1908.03557, 2019.

[27] Y.-C. Chen *et al.*, 'UNITER: UNiversal Image-TExt Representation Learning', in *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXX*, Glasgow, United Kingdom, 2020, pp. 104–120.

[28] M. Conover *et al.*, 'Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM', 2023. [Online]. Available: https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm. [Accessed: 18-May-2025].

[29] P. Rajpurkar, R. Jia, and P. Liang, 'Know What You Don`t Know: Unanswerable Questions for SQuAD', in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2018, pp. 784–789.

[30] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, 'TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension', in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017.

[31] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, 'Bleu: a Method for Automatic Evaluation of Machine Translation', in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[32] C.-Y. Lin, 'ROUGE: A Package for Automatic Evaluation of Summaries', in *Text Summarization Branches Out*, 2004, pp. 74–81.

[33] S. Banerjee and A. Lavie, 'METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments', in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 2005, pp. 65–72.

[34] T.-Y. Lin et al., "Microsoft Coco: Common Objects in Context," Lecture Notes in Computer Science, pp. 740–755, 2014. doi:10.1007/978-3-319-10602-1_48.

[35] R. Krishna et al., “Visual Genome: Connecting Language and Vision Using Crowdsourced Dense Image Annotations,” Int J Comput Vis, vol. 123, no. 1, pp. 32–73, Feb. 2017, doi: 10.1007/s11263-016-0981-7.

[36] K. Han *et al.*, “A survey on Vision Transformer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 87–110, Jan. 2023. doi:10.1109/tpami.2022.3152247

[37] C. Raffel *et al.*, ‘Exploring the limits of transfer learning with a unified text-to-text transformer’, *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.

[38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2019 Conference of the North*, pp. 4171–4186, Jun. 2019. doi:10.18653/v1/n19-1423

[39] Z. Tu, ‘Research on the Application of Layered Architecture in Computer Software Development’, *Journal of Computing and Electronic Information Management*, 2023.

[40] J. Chen *et al.*, ‘We Still Don’t Have Secure Cross-Domain Requests: an Empirical Study of CORS’, in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1079–1093.

[41] U. Tailor and P. Patel, ‘A survey on comparative analysis of horizontal scaling and vertical scaling of cloud computing resources’, *Int J Sci Adv Res Technol*, vol. 2, no. 6, pp. 2395–1052, 2016.

[42] C. Slingerland, “Horizontal vs. vertical scaling: Which should you choose?,” CloudZero, https://www.cloudzero.com/blog/horizontal-vs-vertical-scaling/ (accessed May. 18, 2025).